Análisis de información generada en las plataformas educativas a partir de herramientas de Inteligencia Artificial

Universidad Internacional de Valencia

00 Mes 2026

Titulación:

MIAR – Máster universitario en inteligencia artificial

Curso académico

2025 – 2026

Alumno/a:

Michael Laudrup Luis González

Directora de TFM: Irma Zoraida Sanabria Cárdenas

De:
Planeta Formación y Universidades

Índice

Resumen 9

Abstract 10

1. Introducción 11

2. Objetivos 12

2.1. Objetivo General 12

2.2. Objetivos Específicos 12

3. Marco teórico y estado del arte 14

3.1. Marco teórico 14

3.1.1. Minería de Datos Educativos (EDM) y Learning Analytics. 14

3.1.2. Heterogeneidad de datos y estándares 15

3.1.3. Deserción frente a bajo rendimiento 16

3.1.4. Aprendizaje No Supervisado: Descubrimiento de Patrones (Clustering) 17

3.1.5. Aprendizaje Supervisado y Modelos de Ensamble 18

3.1.6. Síntesis del marco teórico 19

3.2. Estado del arte. 20

3.2.1. Aprendizaje no supervisado en EDM. 20

3.2.2. Aprendizaje supervisado en EDM. 21

3.2.3. El enfoque híbrido. 23

3.2.4. Inteligencia Artificial Explicable (XAI) en Educación. 24

3.2.5. Síntesis del estado del arte 25

4. Planteamiento del problema y metodología 26

4.1. Planteamiento del problema. 26

4.2. Solución propuesta. 27
4.2.1. Ventanas de tiempo. 27
4.2.2. Reducción de dimensionalidad y Aprendizaje no supervisado. 28
4.2.3. Modelado secuencial con Transformer. 30
4.2.4. IA Explicativa. 31
4.2.5. Ecosistema completo 31
4.3. Metodología. 32
4.3.1. Metodología CRISP-DM. 32
4.3.2. Definición de tareas. 33
4.3.3. Planificación. 36
5. Desarrollo del proyecto. 37
5.1. Comprensión del negocio. 37
5.2. Análisis exploratorio de datos (EDA). 39
5.3. Preprocesamiento de datos. 45
5.3.1. Preprocesamiento para EDA. 45
5.3.2. Preprocesamiento para Autoencoder y clustering. 46
5.3.3. Preprocesamiento para red neuronal Transformer. 47
5.4. Ingeniería de características. 48
5.4.1. Fase I: Características para Autoencoder. 49
5.4.2. Fase II: Características para Transformers. 51
5.5. Modelado. 53
5.5.1. Autoencoder. 53
5.5.2. Embeddings. 56
5.5.3. Clustering. 57

5.5.4. Transformer I: Modelado.....62

5.5.5. Transformer I: Estrategia del entrenamiento y configuración de clases. 64

5.6. Evaluación.....66

5.6.1. Evaluación de Autoencoder.....67

5.6.2. Evaluación de Clustering.....67

5.6.3. Evaluación del Transformer.....68

5.7. Dockerización y Pipelines Automatizados.....70

6. Resultados Preliminares.....73

6.1. Resultados de Autoencoder.....73

6.1.1. Fidelidad de reconstrucción.....73

6.1.2. Representaciones vectoriales (Embeddings).....78

6.2. Resultados de clustering.....79

6.3. Resultados en Transformer.....82

6.3.1. Clasificación binaria: Aprobar vs Abandonar.....82

6.3.2. Otros tipos de clasificación.....83

6.4. IA Explicable (XAI).....85

6.4.1. IA Explicativa en el clustering.....85

7. Referencias.....88

Apéndice I: Repositorio de Código y Reproducibilidad.....91

Apendice II: Tablas de características.....94

Características del autoencoder.....94

Apéndice III: Métricas de entrenamiento en Transformers.....97

Anexos I: Estructura y diccionario de datos OULAD.....99

Índice de ilustraciones

Ilustración 2 Diagrama de Deep Clustering Network (DCN) https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=10047866.....29

Ilustración 3 Diagrama representativo de la arquitectura del proyecto.....31

Ilustración 4 Cronograma de tareas definidas. Elaboración propia 36

Ilustración 5 Distribución de estudiantes por resultado académico. Autoría propia - Apéndice 1 - Notebook I 41

Ilustración 6 Análisis comparativo de de interacciones: Ruido diario frente a señal semanal agregada. Autoría propia - Apendice I - Notebook I.....41

Ilustración 7 Contraste entre riqueza y resultados académicos - Autoría Propia - Apéndice I - Notebook I.....42

Ilustración 8 Comparación de interacciones por curso. Autoría propia - Apéndice I - Notebook I 43

Ilustración 9 Relación entre diferentes métricas y resultados académicos. Apéndice 1 - Notebook I.....43

Ilustración 10 Relación entre nivel educativo y resultados académicos Apéndice 1 – Notebook I 44

Ilustración 11 Comparación BIC/AIC en PCA Autoría propia - Apéndice I - Notebook 4 58

Ilustración 12 Evolución BIC/AIC con AE - Autoría propia - Apéndice I - Notebook 4 58

Ilustración 13 Comparativa de resultados en diferentes métricas entre AE y PCA.....61

Ilustración 14 Evolución de huber loss durante entrenamiento de autoencoder 74

Ilustración 15 Evolución de perdidas en la fase de entrenamiento conjunto ...75

Ilustración 16 Balance entre reconstrucción y objetivo de clustering en fase conjunta.....77

Ilustración 17 Representación de datos de estudiantes en el espacio original, latente y reconstrucción - Autoría Propia - Apéndice I - Reportes -AE.....78
Ilustración 18 Agrupaciones de estudiantes por grupos 80
Ilustración 19 Distribución de resultados académicos por perfil en la semana 20 (Validación y Test).....81
Ilustración 20 Matriz Z-Effect de las 30 variables más discriminativas por clúster (W25) 85

Índice de tablas

Tabla 1 Métricas obtenidas en clasificación binaria (Éxito vs Abandono) 82

Tabla 2 Ficha técnica de repositorio código fuente 91

Tabla 3 Características de entrada al autoencoder 96

Tabla 4 Métricas de entrenamiento del transformer clase binaria (Exito vs Fracaso) 97

Tabla 5 Resultados del transformer en clasificación binaria. 97

Tabla 6 Descripción de datos de estudiantes 99

Tabla 7 Descripción de entrega de actividades 100

Tabla 8 Descripción datos de interacción de estudiantes 101

Resumen

Abstract

1. Introducción

La digitalización de los procesos de enseñanza-aprendizaje ha situado a los sistemas de gestión del aprendizaje (LMS) como infraestructuras centrales para organizar y monitorizar la actividad académica. En este contexto, Moodle (Modular Object-Oriented Dynamic Learning Environment) destaca por su adopción extendida y flexibilidad, especialmente en entornos presenciales y a distancia (Gamage, Ayres, & Behrend, 2022)- Esta adopción genera una huella digital detallada: cada interacción del estudiante —accesos, participación, entregas o navegación— queda registrada en forma de *logs*, útiles para analizar comportamientos y modelar fenómenos como retención o carga de trabajo (Soepriyanto, Nugroho, Nahri, Kesuma, & Setiasih, 2025). Sin embargo, su explotación pedagógica sigue siendo limitada: el profesorado carece de herramientas operativas para identificar tempranamente perfiles de aprendizaje y alumnado en riesgo, y muchas decisiones se toman cuando la intervención ya es tardía. En consecuencia, el LMS tiende a funcionar como una "caja negra" difícil de traducir en acciones docentes, lo que refuerza la necesidad de sistemas inteligentes que conviertan los *logs* en señales interpretables y útiles para la prevención del fracaso académico (Khosravi, y otros, 2022)

Este Trabajo de Fin de Máster (TFM) aborda esta problemática mediante técnicas de Inteligencia Artificial (IA) y Minería de Datos Educativos (EDM), orientadas a extraer conocimiento accionable a partir de datos de interacción en LMS, con especial atención a la detección temprana del riesgo académico. La memoria se organiza en ocho capítulos, desde los objetivos y el marco teórico hasta la metodología, el desarrollo, los resultados y los anexos.

2. Objetivos

Tras identificar la brecha existente entre la recolección masiva de datos en entornos virtuales y su limitado aprovechamiento pedagógico, este capítulo formaliza los objetivos que guían la investigación. Para ello, se definen metas que equilibran la capacidad predictiva de los modelos híbridos con la transparencia necesaria para su aplicación en el contexto docente.

2.1. Objetivo General

Diseñar, implementar y evaluar una arquitectura híbrida de aprendizaje profundo aplicada a datos de interacción generados en un LMS (dataset OULAD), con el fin de detectar de forma temprana el riesgo de fracaso y abandono académico, integrando técnicas de aprendizaje no supervisado, modelado secuencial con transformers y métodos de inteligencia artificial explicable que permitan generar conocimiento pedagógicamente accionable.

2.2. Objetivos Específicos

- Analizar la evolución del estado del arte en minería de datos educativos, contrastando las limitaciones de los métodos lineales clásicos (PCA, Regresión) frente a las capacidades de las arquitecturas profundas (deep Learning) para modelar la complejidad del comportamiento estudiantil.
- Consolidar y preprocesar el conjunto de datos OULAD, transformando los registros brutos de interacción en secuencias temporales que permitan modelar la evolución del comportamiento del estudiante en el LMS, superando el uso de métricas estáticas y facilitando su tratamiento mediante arquitecturas Transformer.

- Implementar una arquitectura de Autoencoders (Aprendizaje de Representaciones) para transformar los datos de alta dimensionalidad en un espacio latente comprimido, capaz de capturar relaciones no lineales que escapan a técnicas tradicionales como el PCA.

- Aplicar técnicas de aprendizaje no supervisado mediante algoritmos de clustering sobre distintas representaciones comprimidas de los datos —espacio latente aprendido por Autoencoder y espacio reducido mediante PCA— con el objetivo de segmentar la población estudiantil e identificar arquetipos de aprendizaje emergentes. Por razones de concisión, en esta memoria se presentan únicamente los resultados obtenidos con el modelo de Mixturas Gaussianas (GMM), mientras que el análisis comparativo con otros algoritmos, como K-Means, se documenta en los notebooks de exploración.

- Implementar modelos de predicción secuencial basados en Transformers (Aprendizaje Supervisado), aprovechando el mecanismo de Autoatención (*Self-Attention*) para detectar dependencias a largo plazo en el curso.

- Desplegar una estrategia de Explicabilidad Dual (XAI) para mitigar el problema de la "caja negra", combinando explicabilidad intrínseca mediante la visualización de mapas de atención y explicabilidad agnóstica mediante valores SHAP, con el fin de identificar los momentos clave del curso y cuantificar la contribución individual y global de las variables en la predicción del riesgo académico.

- Diseñar un marco de "Explicaciones Accionables", traduciendo los hallazgos técnicos (pesos de atención y valores SHAP) en recomendaciones pedagógicas concretas que permitan al docente realizar intervenciones personalizadas.

3. Marco teórico y estado del arte

La construcción de un sistema predictivo robusto exige un sólido anclaje en la literatura científica y pedagógica actual. Este capítulo establece los cimientos teóricos de la Minería de Datos Educativos (EDM) y la Analítica de Aprendizaje (LA), revisando las investigaciones de los últimos cinco años que sustentan el paso hacia el deep learning.

A través de este análisis, se justifica la adopción de arquitecturas híbridas —basadas en Autoencoders, técnicas de clustering y Transformers— como la solución óptima para capturar la complejidad dinámica del comportamiento estudiantil en entornos virtuales. Asimismo, se posiciona la Inteligencia Artificial Explicable (XAI) como el puente necesario para transformar estos modelos complejos en herramientas pedagógicas transparentes, permitiendo al docente realizar intervenciones fundamentadas y accionables.

3.1. Marco teórico

A continuación, se desarrolla el marco conceptual que define la Minería de Datos Educativos (EDM) y la Analítica de Aprendizaje (LA), para posteriormente analizar el estado del arte de los últimos cinco años. Esta revisión sistemática permite identificar las limitaciones de los métodos tradicionales y justifica la elección de arquitecturas híbridas y técnicas de explicabilidad (XAI) como respuesta a la complejidad del comportamiento estudiantil en plataformas como Moodle.

3.1.1. Minería de Datos Educativos (EDM) y Learning Analytics.

La minería de datos educativos (EDM, Educational Data Mining) y la analítica del aprendizaje (LA, Learning Analytics) son técnicas que utilizan el

análisis de datos para mejorar los procesos educativos. Para ello se extraen patrones y tendencias del rendimiento, el comportamiento y las interacciones de los estudiantes. Estas disciplinas optimizan los procesos de aprendizaje, identifican a los estudiantes con sus dificultades y logran personalizar las experiencias de aprendizaje (Instituto Andaluz Interuniversitario en Ciencia de Datos e Inteligencia Computacional (DASCI), s.f.).

Para materializar estas promesas, es fundamental comprender el origen de la información. Como se detalla en la revisión fundamental sobre EDM publicada en la revista *Modelling*, el primer paso crítico en cualquier ciclo de minería de datos es la recolección de registros provenientes de los "entornos educativos" donde interactúan los estudiantes (Papadogiannis, Wallace, & Karountzou, 2024). Estos entornos, principalmente los Sistemas de Gestión de Aprendizaje (LMS), actúan como la fuente primaria que alimenta los algoritmos.

3.1.2. Heterogeneidad de datos y estándares

En la actualidad existen una gran diversidad de instituciones educativas con plataformas educativas en línea. Cada una de estas plataformas tiene su propio sistema de gestión del aprendizaje, es decir, su propia manera de recolectar datos de los estudiantes, de estructurar la información y de delimitar qué tipo de datos tienen mayor relevancia sobre otros, así como la relación entre estos. Esta falta de estandarización en la captura y estructuración de los datos plantea un desafío metodológico significativo: la dificultad para replicar estudios y generalizar hallazgos entre distintos entornos tecnológicos (Gašević, Dawson, Rogers, & Gasevic, 2016).

A consecuencia de esta situación, se vuelve necesario establecer un marco de referencia común que garantice la validez externa y la comparabilidad de los resultados (benchmarking). En este contexto, el conjunto de datos OULAD

(Open University Learning Analytics Dataset) se ha consolidado como un conjunto de datos de referencia ampliamente utilizado en la investigación sobre Minería de Datos Educativos. Este dataset destaca no solo por su amplia adopción en la literatura científica, sino por su riqueza multidimensional y su riguroso proceso de tratamiento y certificación garantizando la anonimidad de los datos. Integra datos demográficos, resultados de evaluación y registros detallados de interacción (logs) generados en el Entorno Virtual de Aprendizaje (VLE) de la Open University durante los cursos académicos 2013 y 2014. (Kuzilek, Hlosta, & Zdrahal, 2017)

Si bien han surgido conjuntos de datos más recientes en los últimos años, su idoneidad para la presente investigación es limitada. Gran parte de los datasets actuales (procedentes de competiciones tipo Kaggle o plataformas MOOC específicas) presentan inconvenientes críticos: o bien se trata de datos sintéticos generados artificialmente para sortear regulaciones de privacidad —lo que compromete la fidelidad del comportamiento humano real—, o bien carecen de la documentación académica rigurosa necesaria para sustentar una investigación formal. Asimismo, alternativas masivas como EdNet se centran excesivamente en el trazado de conocimiento (Knowledge Tracing) a nivel de ítem, alejándose de la estructura general de interacción típica de Moodle que este trabajo pretende modelar. Por tanto, OULAD se mantiene como un recurso idóneo para validar modelos predictivos en entornos LMS tradicionales, cuya relevancia continúa siendo reconocida en los inventarios de datos de la literatura científica actual (Papadogiannis, Wallace, & Karountzou, 2024).

3.1.3. Deserción frente a bajo rendimiento

En el ámbito del *Learning Analytics* es crucial diferenciar entre la deserción (dropout) y el bajo rendimiento (failure).

La deserción se define como el cese de la actividad en la plataforma y la desvinculación formal del curso antes de su finalización. Este fenómeno suele estar asociado a factores motivacionales, gestión del tiempo o insatisfacción con el entorno. En el dataset OULAD, esto se representa mediante la etiqueta Withdrawn (retirada). Por el contrario, el bajo rendimiento se refiere a aquellos estudiantes que, completando el ciclo del curso y participando en las evaluaciones, no alcanzan los estándares mínimos de conocimiento, etiquetados como Fail.

Distinguir estos dos fenómenos es fundamental para la modelización computacional, ya que los patrones de interacción que preceden a un abandono (ej. disminución progresiva de accesos) difieren sustancialmente de los patrones de un estudiante que suspende (ej. accesos constantes, pero bajo desempeño en cuestionarios). Un sistema robusto debe ser capaz de diagnosticar ambos riesgos por separado.

3.1.4. Aprendizaje No Supervisado: Descubrimiento de Patrones (Clustering)

El Aprendizaje No Supervisado es una rama de la Inteligencia Artificial donde el modelo trabaja con datos no etiquetados, buscando estructuras ocultas o patrones intrínsecos en la información. En el contexto educativo, su aplicación principal es el clustering o agrupamiento.

A diferencia de la clasificación, donde se le dice al algoritmo "este es un buen alumno", el clustering agrupa a los estudiantes basándose únicamente en la similitud de sus comportamientos (ej. frecuencia de accesos, horarios de conexión, tipos de recursos visitados). Esto permite identificar arquetipos de estudiantes (perfiles) que no son evidentes a simple vista, como "el estudiante

intensivo de fin de semana" o "el estudiante que solo lee foros, pero no participa".

3.1.5. Aprendizaje Supervisado y Modelos de Ensamble

El Aprendizaje Supervisado implica entrenar algoritmos con un conjunto de datos donde se conoce la respuesta correcta (la variable objetivo, en este caso, el resultado final del alumno), para que el modelo aprenda a predecir dicha variable en nuevos estudiantes basándose en su historial. En el contexto de la minería de datos educativos, la evolución de estos modelos ha pasado por tres fases diferenciadas:

- Modelos Lineales: Tradicionalmente, se empleaban técnicas como la Regresión Logística, insuficientes para capturar las relaciones complejas y no lineales del comportamiento estudiantil.
- Modelos de Ensamble (Ensemble Learning): Estudios comparativos recientes sobre el rendimiento estudiantil (Hasan, y otros, 2020) han validado que los algoritmos de ensamble, específicamente Random Forest, superan en precisión a las técnicas clásicas y a las redes neuronales simples cuando se trabaja con datos estructurados. Estos combinan múltiples "árboles de decisión" para manejar el desbalance de clases y determinar la importancia de las variables (Feature Importance). Sin embargo, tal como se evidencia en la metodología empleada por Hasan, y otros (2020), estos modelos dependen de características agregadas (ej. recuento total de visualizaciones), lo que reduce la compleja dinámica temporal del aprendizaje a valores estáticos acumulados, perdiendo el contexto secuencial.
- Deep learning Secuencial y Transformers: Dado que el aprendizaje es un proceso dinámico que evoluciona en el tiempo, el estado del arte más avanzado se ha desplazado hacia redes neuronales capaces de procesar secuencias (Kusumawardani & Alfarozi, 2023).

Inicialmente, las redes recurrentes (RNN y LSTM) permitieron analizar la evolución temporal del alumno. Más recientemente, la arquitectura transformer ha ganado protagonismo en el modelado secuencial. A diferencia de las anteriores, los transformers se basan en mecanismos de autoatención (self-attention), que permiten ponderar la relevancia de cada evento del curso en relación con los demás, independientemente de la distancia temporal entre ellos. De este modo, es posible capturar dependencias a largo plazo (por ejemplo, cómo el desempeño en una tarea temprana puede influir en el abandono en semanas posteriores) (Kusumawardani & Alfarozi, 2023; Kuzilek, Hlosta, & Zdrahal, 2017).

3.1.6. Síntesis del marco teórico

En síntesis, el marco teórico expuesto evidencia que tanto las técnicas no supervisadas como las supervisadas poseen fortalezas complementarias. Mientras que el clustering permite descubrir la estructura subyacente de los datos sin sesgos previos, los modelos supervisados capitalizan esa información para realizar predicciones concretas. Esta complementariedad teórica sugiere que la integración de ambas ramas —en un enfoque híbrido— constituye la arquitectura idónea para abordar la complejidad del comportamiento estudiantil en plataformas como Moodle, premisa que guiará la revisión de la literatura y la propuesta metodológica de este trabajo.

3.2. Estado del arte.

La literatura científica reciente en minería de datos educativos ha evolucionado desde la aplicación aislada de algoritmos básicos hacia arquitecturas cada vez más complejas orientadas a la personalización. Para contextualizar la contribución de este TFM, esta sección analiza las investigaciones más relevantes de los últimos cinco años que han utilizado el dataset OULAD. El análisis se estructura focalizándose en la evolución metodológica: partiendo de los enfoques puramente exploratorios (no supervisados) y predictivos (supervisados), hasta llegar a las propuestas híbridas más vanguardistas. Asimismo, se identifican las limitaciones recurrentes en los estudios actuales —específicamente en la reducción de dimensionalidad y la interpretabilidad— que justifican la necesidad de explorar técnicas no lineales de Deep Learning.

3.2.1. Aprendizaje no supervisado en EDM.

En el contexto específico del dataset OULAD, la investigación más reciente no solo valida la eficacia de las técnicas de clustering, sino que las posiciona como herramientas indispensables para la personalización educativa. Un ejemplo es el estudio publicado por El Ghali, Atouf, El Guemmat, Broumi, & Talea (2025), quienes profundizaron en la segmentación estratégica de estudiantes utilizando este conjunto de datos. Su investigación contrastó algoritmos basados en vecindad (KNN) frente al clustering jerárquico, empleando PCA (Análisis de Componentes Principales) para gestionar la alta dimensionalidad de las variables demográficas y de interacción.

Los resultados presentados por El Ghali et al. marcan el estándar del estado del arte actual: la aplicación del PCA no solo redujo el tiempo de cómputo en un 60%, sino que, combinado con clustering jerárquico, permitió alcanzar métricas de calidad excepcionales. Mediante esta metodología, lograron identificar cuatro

arquetipos de estudiantes, revelando perfiles complejos como aquellos con "alta participación, pero bajo rendimiento" o patrones de compromiso erráticos (El Ghali, Atouf, El Guemmat, Broumi, & Talea, 2025). Estos hallazgos son relevantes para la presente investigación, ya que sugieren que el dataset OULAD contiene estructura y regularidades aprovechables.

3.2.2. Aprendizaje supervisado en EDM.

La evolución de los modelos predictivos aplicados a OULAD muestra una transición desde clasificadores lineales hacia métodos de ensamble y, más recientemente, hacia enfoques secuenciales de aprendizaje profundo. Este desplazamiento es coherente con la propia naturaleza del dataset OULAD.

En tareas con variables tabulares agregadas, algoritmos de gradient boosting como XGBoost se emplean con frecuencia como línea base competitiva; por ejemplo, Firat (2025) reporta mejores resultados de XGBoost frente a Random Forest en su configuración experimental, destacando además la dependencia de estos modelos respecto a la ingeniería manual de características. En este sentido, parte de la literatura enfatiza que la ganancia de rendimiento en ensambles se apoya en “cómo” se agregan los logs (por semana/día, por tipo de actividad, por curso) y en la selección de variables, lo que puede limitar la captura de patrones temporales finos cuando se reduce la secuencia a totales acumulados.

Para incorporar la dimensión temporal de los registros del LMS, trabajos recientes han evaluado modelos secuenciales: Torkhani & Rezgui (2025) obtienen alrededor de un 83% de accuracy con LSTM, superando a varios enfoques clásicos en su comparación. Este resultado es consistente con la línea de trabajos que tratan los clics/interacciones como series temporales (p. ej., agregación semanal) para anticipar riesgo académico antes de finalizar el curso,

priorizando el "cuándo" además del "cuánto" interactúa el estudiante. Además, otros enfoques recientes amplían el foco a escenarios multiclase (pass, fail, withdrawn, distinction) y muestran que redes neuronales pueden mejorar la predicción temprana cuando integran diversas fuentes (demografía, evaluaciones y clickstream) con preprocesado y feature engineering más exhaustivos (Junejo, y otros, 2024).

En esta misma línea, estudios como Kusumawardani & Alfarozi (2023) exploran transformer encoders sobre OULAD para predicción secuencial (diaria/semanal), reportando resultados competitivos y ventajas prácticas asociadas a la atención y la paralelización del entrenamiento. De forma complementaria, investigaciones recientes sobre OULAD centradas específicamente en abandono (withdrawal) también han propuesto variantes transformer y reportan un rendimiento alto y estable en predicción temprana cuando se usan secuencias semanales de actividad (Luo, Yu, Xie, Wu, & Xu, 2024). Este tipo de evidencias refuerza la idea de que, cuando se preserva la estructura secuencial de los logs, la autoatención puede competir (y en algunos diseños superar) a enfoques recurrentes en tareas de detección temprana de riesgo.

Más allá de la eficiencia, una posible ventaja de los transformers para este TFM es su utilidad interpretativa como señal complementaria: los mecanismos de autoatención generan pesos que permiten analizar qué partes de la secuencia (por ejemplo, interacciones en semanas concretas) han sido más relevantes para la predicción. No obstante, la literatura suele recomendar tratar la atención como una señal interpretativa "auxiliar" y complementarla con métodos de explicabilidad agnósticos al modelo para obtener justificaciones más robustas. En consecuencia, este TFM propone no solo evaluar modelos transformer en términos de rendimiento, sino también analizar patrones de atención y complementarlos con técnicas agnósticas como SHAP para reforzar la

explicabilidad (XAI) del sistema híbrido, facilitando que el docente comprenda el "porqué" del riesgo detectado.

3.2.3. El enfoque híbrido.

Los modelos híbridos representan la vanguardia en la minería de datos educativos, combinando técnicas no supervisadas y supervisadas en una arquitectura secuencial. A diferencia de los modelos monolíticos tradicionales, el enfoque híbrido segmenta primero la población mediante clustering para luego entrenar predictores especializados en cada perfil.

Esta estrategia ha sido validada empíricamente por Al-Tameemi et al. (2024) quienes demostraron que el uso de clústeres previos mejora significativamente la precisión de la clasificación en entornos educativos. No obstante, existe un patrón recurrente en la literatura de vanguardia, observado tanto en Al-Tameemi et al. como en El Ghali et al. (2025): la omnipresencia del PCA como técnica estándar de reducción de dimensionalidad, lo que implica una presuposición de que la relación entre los datos es lineal.

Dado que el aprendizaje humano es un proceso complejo y dinámico, la asunción de linealidad puede resultar limitante en ciertos contextos y simplificar relaciones potencialmente más ricas en los datos. Por ello, este TFM plantea complementar el enfoque basado en PCA con una exploración de arquitecturas de autoencoders (Deep Learning) como alternativa no lineal de reducción de dimensionalidad. El objetivo es evaluar empíricamente si estas representaciones latentes son capaces de capturar dependencias que PCA no modela de forma directa y, en caso de aportar valor, utilizarlas como base para los modelos predictivos posteriores.

3.2.4. Inteligencia Artificial Explicable (XAI) en Educación.

Tal como se adelantó al finalizar el análisis sobre los modelos supervisados y la arquitectura Transformer, la búsqueda de una mayor precisión predictiva conlleva la creación de "cajas negras", modelos con estructuras complejas que no son fácilmente interpretables (Khosravi, y otros, 2022).En el ámbito educativo, un modelo que predice el fracaso con un 90% de acierto pero que no explica las causas es pedagógicamente estéril, ya que impide al docente diseñar una intervención correctiva fundamentada. La literatura define esto como la necesidad de generar "explicaciones accionables" (actionable explanations), entendidas como datos que permiten establecer un procedimiento correctivo o bucle de retroalimentación para un conjunto de acciones (Khosravi, y otros, 2022).

Estudios recientes coinciden en que la adopción efectiva de la Minería de Datos Educativos depende de la confianza del usuario final, siendo la explicabilidad un factor clave para incrementarla. Sin embargo, los modelos avanzados de *Deep Learning*, como LSTM o Autoencoders, operan mediante transformaciones no lineales complejas que dificultan la comprensión de la relación entre entrada y salida, incluso para expertos, debido a la geometría de sus espacios de alta dimensión (Sejnowski, 2020). Esta opacidad limita la capacidad del docente para interpretar las causas subyacentes de una predicción de riesgo.

Para mitigar esta problemática sin renunciar a la potencia de los modelos no lineales, este TFM adopta una estrategia de explicabilidad dual, alineada con el marco de trabajo XAI-ED:

- Explicabilidad Intrínseca (Attention Maps): Aprovechando la arquitectura Transformer seleccionada, se extraerán y visualizarán las matrices de pesos de atención (Self-Attention Weights). Estos mapas

permiten observar directamente en qué momentos del curso o actividades específicas se "fijó" el modelo para determinar el riesgo de un estudiante Este enfoque sigue las recomendaciones de complementar modelos complejos con componentes visuales interpretables para garantizar la transparencia (Khosravi, y otros, 2022).

- Explicabilidad Agnóstica del Modelo (SHAP): Para garantizar una interpretación global, se utilizarán valores SHAP (Shapley Additive exPlanations). Esta técnica, fundamentada en la teoría de juegos, permite asignar una puntuación de contribución a cada variable, desvelando qué características empujaron la predicción hacia el éxito o el fracaso y distribuyendo equitativamente la importancia entre ellas (Lundberg & Lee, 2017).

3.2.5. Síntesis del estado del arte

La revisión bibliográfica sobre el dataset OULAD pone de manifiesto tres tendencias recurrentes en la literatura reciente: el predominio de técnicas lineales como PCA, lo que sugiere explorar representaciones no lineales mediante Autoencoders; la progresiva adopción de modelos secuenciales, donde las arquitecturas Transformer emergen como una alternativa prometedora para capturar dependencias temporales; y la necesidad transversal de incorporar explicabilidad para garantizar la adopción pedagógica de modelos complejos.

No obstante, estas aproximaciones han sido abordadas de forma aislada, sin evidenciarse una arquitectura que las integre simultáneamente. En este contexto, la combinación de Autoencoders, Transformers y técnicas XAI se plantea como una propuesta híbrida novedosa orientada a cubrir este vacío y a evaluar las ventajas de su integración sinérgica frente a los enfoques fragmentados del estado del arte.

4. Planteamiento del problema y metodología.

Este capítulo presenta el desarrollo técnico del sistema propuesto para la detección temprana del riesgo académico a partir de los datos del dataset OULAD. Tras la fundamentación teórica expuesta en los capítulos anteriores, se formaliza aquí el problema desde una perspectiva computacional y se describe la metodología seguida para su resolución.

En primer lugar, se delimita el planteamiento del problema, identificando las limitaciones actuales en el aprovechamiento pedagógico de los registros generados en los LMS. A continuación, se detalla la solución propuesta, basada en una arquitectura híbrida que combina aprendizaje no supervisado y modelado secuencial.

Finalmente, se expone el marco metodológico adoptado —estructurado bajo el estándar CRISP-DM— así como las fases operativas que guían el preprocesamiento, el entrenamiento de los modelos y la evaluación del sistema, con especial atención a su interpretabilidad y aplicabilidad en el contexto docente.

4.1. Planteamiento del problema.

En la actualidad, la digitalización educativa ha convertido a los Sistemas de Gestión del Aprendizaje (LMS), especialmente Moodle, en infraestructuras críticas que generan un volumen masivo de datos. Cada interacción del estudiante genera una "huella digital" detallada en los archivos de registro o logs (Soepriyanto, Nugroho, Nahri, Kesuma, & Setiasih, 2025).

A pesar de esta abundancia de información, existe una brecha crítica entre la recolección de datos y su aprovechamiento pedagógico real. El problema se desglosa en cuatro ejes fundamentales:

- Opacidad y falta de interpretabilidad: Como se detalló en el marco teórico, los LMS y los modelos predictivos avanzados suelen operar como "cajas negras" (Khosravi et al., 2022), dificultando la extracción de explicaciones claras y pedagógicamente útiles para el profesorado.
- Detección tardía: Falta de herramientas para identificar de forma temprana a estudiantes en riesgo de fracaso o abandono, cuando la intervención ya no es efectiva.
- Limitación en el modelado temporal: Predominio de enfoques basados en métricas estáticas acumuladas que ignoran la naturaleza secuencial del aprendizaje e impiden detectar cambios conductuales indicativos de riesgo.

Se requiere, por tanto, un sistema inteligente que prediga el rendimiento final de manera explicable, transformando los datos brutos en conocimiento accionable para facilitar la intervención del docente.

4.2. Solución propuesta.

Al abordar un problema de detección temprana de riesgo, uno de los principales desafíos metodológicos es evitar la "fuga de datos" (data leakage).

Para que el modelo predictivo sea válido y aplicable a la realidad docente, es indispensable garantizar que solo evalúe la información generada desde el inicio del curso (Día 0) hasta el momento exacto en el que se realiza la predicción.

4.2.1. Ventanas de tiempo.

Esta limitación hace necesario introducir el concepto de ventana de tiempo, un enfoque fundamental para estructurar el análisis en este proyecto. Como ya se observó en el Análisis Exploratorio de Datos (EDA), el registro diario de interacciones presenta un nivel de ruido que dificulta la extracción de patrones claros. Sin embargo, al agregar la actividad de los estudiantes de forma semanal,

el comportamiento se vuelve mucho más estable y tratable. Esta decisión de diseño concuerda con los hallazgos de Kusumawardani y Alfarozi (2023), quienes demostraron empíricamente que los modelos basados en ventanas semanales tienden a superar a los diarios y resultan computacionalmente más eficientes.

A partir de este punto, la temporalidad del problema se modela exclusivamente mediante ventanas semanales acumulativas, desde el inicio del curso hasta la semana W, momento en el que se estima el riesgo de abandono. En la construcción de las variables se garantiza estrictamente la ausencia de información futura, de modo que cada predicción utiliza únicamente los datos disponibles hasta la semana analizada, asegurando el funcionamiento del sistema como una alerta temprana.

En cada ventana de tiempo calculamos métricas acumuladas hasta esa semana (clics en los foros, actividades entregadas, notas acumuladas hasta entonces, etc.) y las juntamos con variables que conocemos desde el día cero del estudiante (rango de edad, índice de riqueza del código postal, interactividad antes del comienzo del curso con la plataforma educativa, etc.)

4.2.2. Reducción de dimensionalidad y Aprendizaje no supervisado.

Una vez obtenida esta información (aproximadamente 58 características), siguiendo una aproximación inspirada en el estudio de El Ghali, Atouf, El Guemmat, Broumi, & Talea (2025), se realiza un agrupamiento (clustering) del estudiantado utilizando exclusivamente la información disponible hasta la semana W, con el objetivo de identificar perfiles de comportamiento diferenciados.

Para optimizar el agrupamiento, se comparan técnicas de reducción de dimensionalidad, concretamente PCA y un autoencoder, con el fin de proyectar

la información a un espacio latente de menor dimensión y potencialmente más separable. En el caso del autoencoder se le penalizará no solo por su capacidad de reconstrucción sino también por su capacidad para clusterizar, más visualmente podemos verlo en la siguiente figura:

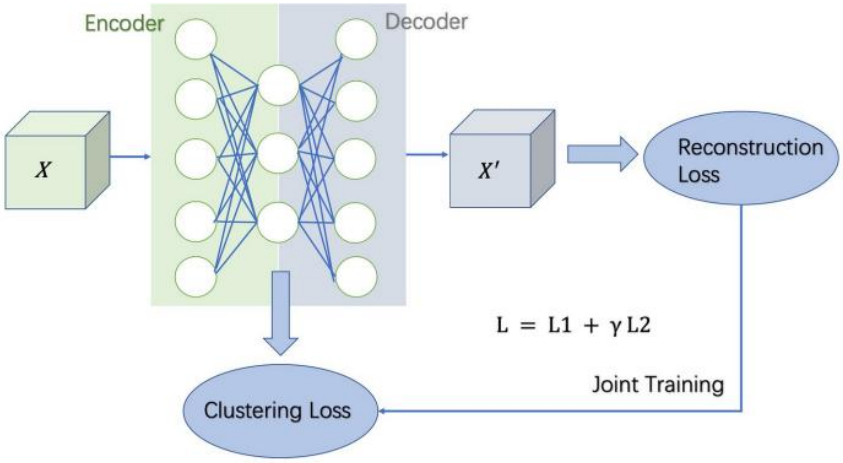

Ilustración 1 Diagrama de Deep Clustering Network (DCN)
https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=10047866

En concreto, se entrenará un único autoencoder (común a todas las ventanas) para aprender una representación comprimida de los datos. Posteriormente, se ajustará un modelo de clustering independiente para cada ventana temporal W, de modo que cada uno esté especializado en agrupar a los estudiantes basándose en su comportamiento acumulado desde el inicio del curso hasta la semana W.

El algoritmo de agrupamiento seleccionado es el Modelo de Mixtura Gaussiana (GMM), cuya naturaleza probabilística se adapta a distribuciones continuas con fronteras difusas entre perfiles. A diferencia del hard clustering, GMM asigna a cada estudiante una distribución de probabilidades, lo que permite alimentar al modelo Transformer posterior con soft labels más informativas y estables que identificadores de clase discretos.

4.2.3. Modelado secuencial con Transformer.

El objetivo principal de la red neuronal Transformer es predecir de forma temprana el riesgo de abandono (*Withdrawn*) o de fracaso académico (*Fail*). La naturaleza del problema predictivo varía en función de la ventana temporal analizada: en etapas tempranas del curso (p. ej., semanas 5, 10 o 15), la escasez de evaluaciones consolidadas limita la predicción del suspenso, dando lugar a un escenario de clasificación binaria (aprobar frente a abandonar). A medida que avanza el curso y se dispone de mayor evidencia, el problema evoluciona hacia una clasificación multiclase (aprobar, abandonar o suspender). Para gestionar esta dinámica, se diseñan Transformers independientes, cada uno especializado en realizar predicciones utilizando exclusivamente la información disponible hasta su ventana temporal correspondiente.

Aunque cada Transformer se entrena de forma específica para su ventana temporal, todos forman parte de un mismo pipeline arquitectónico, compartiendo el preprocesamiento, las representaciones latentes generadas en la fase de reducción de dimensionalidad y la integración con el módulo de clustering. De este modo, no se trata de modelos aislados, sino de componentes especializados dentro de un ecosistema metodológico único.

Para alimentar estos modelos, se procesarán secuencias temporales que capturen la evolución semanal de la interactividad del estudiante en la plataforma. A esta información secuencial se le integrarán las características derivadas de la fase previa de clustering, concretamente los vectores probabilísticos de pertenencia (soft labels) generados por el GMM. Mediante esta arquitectura híbrida, se busca demostrar empíricamente que el enriquecimiento de los datos secuenciales con representaciones latentes de comportamiento mejora sustancialmente el rendimiento predictivo del Transformer.

4.2.4. IA Explicativa.

Para dotar de transparencia al sistema y resolver el problema de interpretabilidad mencionado, se integrarán técnicas de Inteligencia Artificial Explicable (XAI). El propósito es dotar de transparencia a las predicciones generadas tanto en la fase de clustering como en la clasificación del Transformer, transformando los resultados matemáticos en explicaciones pedagógicas accionables para el docente.

4.2.5. Ecosistema completo

En la siguiente figura se presenta una visión global del ecosistema metodológico del proyecto, integrando el flujo de información desde las fuentes de datos hasta la generación de predicciones y recomendaciones pedagógicas. El diagrama sintetiza las distintas capas del sistema —preprocesamiento, aprendizaje no supervisado, modelado secuencial y XAI— y sus interacciones, mostrando la contribución coordinada de cada componente a la detección temprana del riesgo académico.

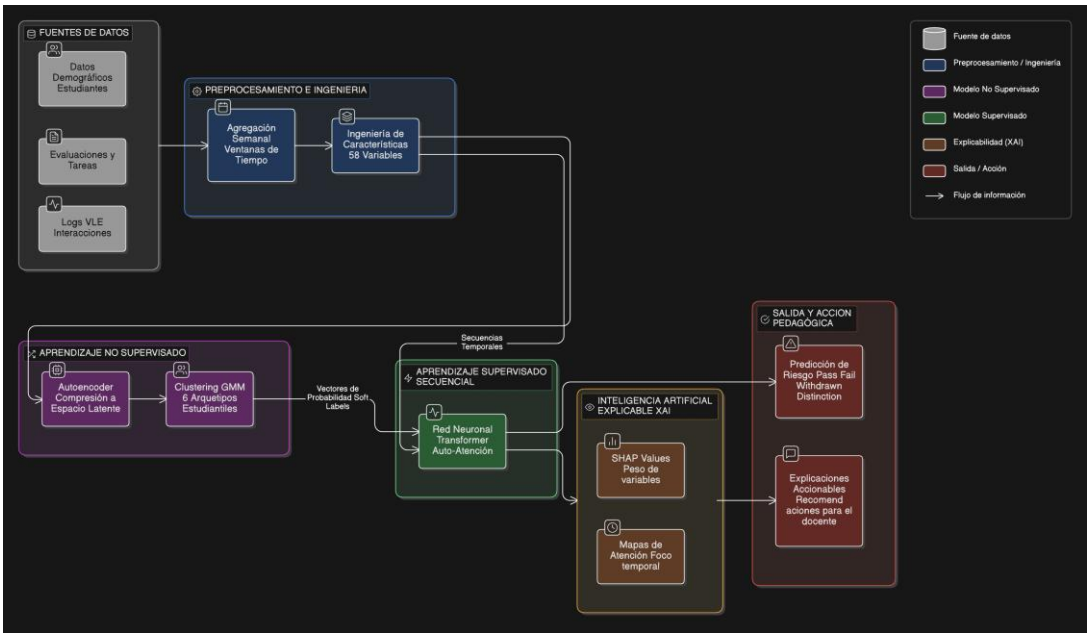

Ilustración 2 Diagrama representativo de la arquitectura del proyecto

4.3. Metodología.

La implementación de un sistema predictivo basado en *Deep Learning* dentro del contexto educativo requiere un enfoque estructurado que armonice los requisitos técnicos con las necesidades docentes. En este subapartado se detalla el marco metodológico adoptado, el cual permite transitar de manera sistemática desde la fase de comprensión de los datos brutos hasta la generación de acciones pedagógicas concretas y explicables.

Para garantizar la rigurosidad científica y la replicabilidad de la investigación, se describe a continuación el estándar procedimental seguido (CRISP-DM), la jerarquía de las tareas operativas diseñadas para este estudio junto con las tareas específicas a realizar y la hoja de ruta temporal que guía el desarrollo del proyecto.

4.3.1. Metodología CRISP-DM.

Para transformar los datos brutos en conocimiento útil, este proyecto adopta la metodología iterativa CRISP-DM (Wirth & Hipp, 2000). A diferencia de un proceso lineal CRISP-DM es cíclico e iterativo: permite volver atrás y refinar lo aprendido en cada etapa para que el resultado final sea lo más preciso posible. Para este estudio, el ciclo de vida se ha personalizado en seis grandes bloques:

- Comprensión del Negocio: Definir la necesidad pedagógica: ¿por qué abandonan los alumnos y qué información necesita el profesor para ayudarles a tiempo? * Exploración de Datos (EDA): Analizar la "huella digital" del estudiante en Moodle para comprender la información disponible.

- Preparación de Datos: Transformar los clics diarios en secuencias que cuentan la historia del estudiante semana a semana, siguiendo la tendencia actual de analíticas de aprendizaje (Soepriyanto, Nugroho, Nahri, Kesuma, & Setiasih, 2025).
- Modelado: Aplicar modelos avanzados (Deep Learning) capaces de aprender patrones de comportamiento que no son obvios a simple vista.
- Evaluación y Explicabilidad (XAI): No basta con que el modelo acierte; se integran técnicas explicables para traducir las decisiones matemáticas de la máquina en motivos claros para el docente.
- Despliegue: Convertir las predicciones en recomendaciones pedagógicas aplicables en un entorno real.

Adicionalmente, cabe subrayar que, para este TFM, este bloque no se ha concebido como una serie de pasos manuales, sino como un pipeline de datos automatizado. Esta decisión metodológica garantiza que cualquier transformación (normalización, imputación o ingeniería de variables) se aplique de forma estricta e idéntica a los conjuntos de entrenamiento, validación y test, eliminando el riesgo de "fuga de datos" y asegurando la reproducibilidad total del experimento.

Definida la metodología de trabajo, el siguiente paso consiste en delimitar el alcance del problema técnico y pedagógico. No basta con saber 'cómo' vamos a trabajar (CRISP-DM), sino que debemos precisar 'qué' estamos intentando resolver y cuáles son los hitos específicos que permitirán alcanzar el éxito del modelo.

4.3.2. Definición de tareas.

Para alcanzar los objetivos propuestos, el proyecto se ha desglosado en una serie de hitos técnicos y operativos. Estas tareas no solo cubren el desarrollo del

código, sino también la fundamentación teórica y la transferencia de resultados al ámbito pedagógico:

Fase I: Fundamentación y contextualización

- Comprensión del ecosistema (Business Understanding): Análisis del dominio de la Minería de Datos Educativos (EDM) y la Analítica de Aprendizaje para alinear el modelo con las necesidades docentes.
- Definición de objetivos: Formalización del problema a resolver, estableciendo las métricas de éxito y los límites del sistema predictivo.
- Investigación del estado del arte: Revisión sistemática de literatura científica de los últimos cinco años para identificar arquitecturas de vanguardia y evitar redundancias en la investigación.

Fase II: Gestión y Exploración de Datos

- Adquisición de la fuente de datos: Selección y descarga del conjunto de datos OULAD, por su representatividad en interacciones tipo Moodle.
- Pre-procesamiento y Limpieza: Tratamiento de valores nulos, normalización de variables y filtrado de registros inconsistentes para asegurar la calidad de la entrada.
- Análisis Exploratorio de Datos (EDA): Identificación de patrones visuales, correlaciones y desequilibrios de clase (ej. tasa de abandono frente a suspensos).

Fase III: Ingeniería de Características y Modelado Híbrido.

- Ingeniería de Características I (Transformación Temporal): Conversión de los *logs* de interacción brutos en secuencias temporales estructuradas por semanas de curso.

- Modelado No Supervisado (Clustering): Implementación de algoritmos para descubrir arquetipos de estudiantes de forma automática.
- Ingeniería de Características II (Enriquecimiento): Integración de las etiquetas de los clústeres en el conjunto de entrenamiento para potenciar la capacidad de discriminación del modelo.
- Desarrollo del Modelo Supervisado: Diseño y entrenamiento de una red neuronal basada en la arquitectura Transformer, optimizada para capturar dependencias secuenciales a largo plazo.

Fase IV: Evaluación y Explicabilidad

- Evaluación del desempeño: Cálculo de métricas de rendimiento (Accuracy, F1-Score, Precisión) para validar la robustez de las predicciones.
- Implementación de Inteligencia Artificial Explicable (XAI): Aplicación de técnicas de interpretabilidad para desglosar el peso de cada variable y visualizar los mapas de atención del modelo.
- Acciones Pedagógicas (Transferencia): Traducción de los hallazgos técnicos en recomendaciones concretas para la intervención docente personalizada.

4.3.3. Planificación.

A continuación, se plasman en un diagrama de tipo Gantt la hoja de ruta cronológica y la distribución de esfuerzos diseñada para este trabajo. Este cronograma organiza temporalmente las cuatro fases operativas descritas anteriormente, estableciendo una secuencia lógica que garantiza el cumplimiento de los objetivos técnicos y pedagógicos dentro del plazo académico establecido.

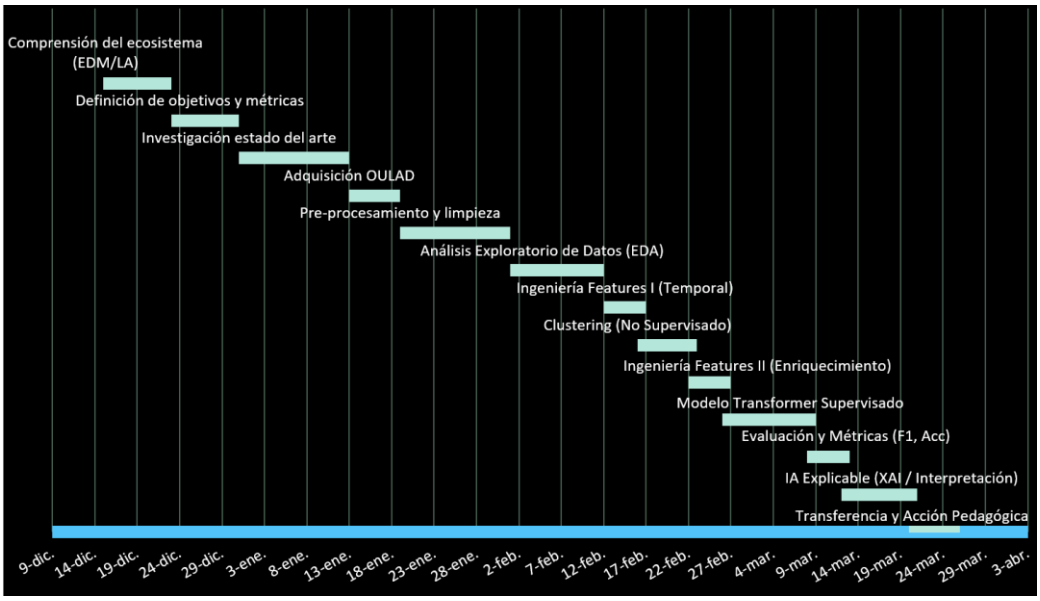

Ilustración 3 Cronograma de tareas definidas. Elaboración propia

5. Desarrollo del proyecto.

La implementación técnica de la arquitectura propuesta se ha llevado a cabo siguiendo los estándares de desarrollo de software y ciencia de datos. Cabe destacar que la totalidad del código fuente, los scripts de preprocesamiento y los cuadernos de experimentación se encuentran alojados en un repositorio público de GitHub (Véase Apéndice I: Repositorio de Código y Reproducibilidad). Esta decisión responde a un doble objetivo metodológico:

- Garantizar la trazabilidad y autoría: El historial de *commits* (registro de cambios) actúa como una prueba temporal veraz del progreso y la originalidad del trabajo realizado, así como la autoría de este.
- Optimizar la memoria académica: Para evitar que este documento se extienda innecesariamente con bloques de código, este apartado se centra en describir la lógica, las decisiones arquitectónicas y los flujos de datos más relevantes.

La estructura de este apartado sigue fielmente las fases del ciclo de vida CRISP-DM, detallando cómo se ha ejecutado cada etapa técnica:

5.1. Comprensión del negocio.

El proceso de comprensión del negocio se inició con una investigación exhaustiva sobre fuentes de datos que cumplieran con requisitos de fiabilidad, volumen y representatividad de un entorno LMS real. Tras evaluar diversas alternativas, se seleccionó el conjunto de datos OULAD (Open University Learning Analytics Dataset). La elección de este dataset se fundamenta en su veracidad, anonimizarían rigurosa, volumen representativo y su idoneidad para realizar comparativas (benchmarking) con el estado del arte.

Cabe señalar que la fundamentación teórica del problema, así como el análisis profundo del estado del arte que contextualiza estos datos, se encuentran documentados en detalle en el Capítulo 3 de esta memoria. El análisis estructural del conjunto de datos OULAD se fundamenta en tres dimensiones que alimentarán el modelo híbrido propuesto:

- Dimensión del estudiante: Consolida la información estática y demográfica, actuando como el perfil base de cada alumno matriculado. Ejemplos: El género del estudiante, su región de residencia, el nivel educativo previo, etc.
- Dimensión de actividades: Contiene los resultados en diferentes actividades realizadas durante el curso por parte del alumno. Ejemplos: La calificación obtenida en una tarea (escala 0-100), el tipo de evaluación (si es corregida por un tutor o automática) y la fecha real en la que el alumno realizó la entrega frente a la fecha límite
- Dimensión de comportamiento: Representa el log de actividad diario en el Sistema de gestión del aprendizaje (LMS) y constituye la huella digital que el estudiante deja al interaccionar con la plataforma. Ejemplos: El número de clics o interacciones diarias con materiales de lectura, la participación en foros de discusión, el acceso a cuestionarios o la descarga de recursos específicos del curso.

El diccionario de datos detallado con la descripción exhaustiva de todas las variables que componen estas tres dimensiones puede consultarse en el Anexo I: Estructura y Diccionario de Datos OULAD. Otros aspectos relevantes de la base de datos son:

- Las fechas normalmente están relativizas con respecto al inicio del curso y se expresan en días previos (negativo) o posteriores (positivos).

- Las interacciones no solo se recogen desde el día cero del curso, sino que antes de iniciar el curso, ya se capturan las interacciones que hacen los estudiantes con la plataforma.

La convergencia entre el análisis de la estructura y relación de los datos con el estudio exhaustivo del marco teórico y el estado del arte constituye el núcleo de la fase de comprensión del negocio. Es importante destacar que, bajo el enfoque iterativo de la metodología CRISP-DM, esta comprensión no es estática; por el contrario, se profundiza y matiza en la comprensión del negocio en cada una de las fases sucesivas del proyecto, desde el preprocesamiento hasta la evaluación de los modelos.

Una vez consolidada esta base conceptual y estratégica, se procede a ejecutar el análisis exploratorio de datos (EDA). El propósito de esta etapa es validar empíricamente la estructura y las interacciones de los datos, permitiendo identificar patrones, sesgos o anomalías que fundamentarán las decisiones técnicas de las fases posteriores

5.2. Análisis exploratorio de datos (EDA).

A nivel técnico, la exploración de los datos (EDA) se materializó mediante scripts de análisis en un Jupyter Notebook, concretamente en el archivo I_data_exploration.ipynb. Este cuaderno, cuya ubicación exacta dentro de la estructura del proyecto se detalla en el Apéndice I: Repositorio de Código y Reproducibilidad, permite examinar las distribuciones de las variables y la correlación entre las interacciones y el rendimiento final. Del análisis de datos exploratorio (EDA) realizado en dicho cuaderno, se extrajeron las siguientes conclusiones más relevantes:

- Descarte por ética y directrices del EU AI act (Parlamento Europeo y Consejo de la Unión Europea, 2024): Para hacer un sistema

ético, equitativo y que cumpla la legislación vigente en la unión europea, datos sensibles como género y discapacidad se descartan del estudio.

- Valores nulos: En el conjunto de datos OULAD, los valores nulos poseen un alto significado estructural y rara vez responden a deficiencias de calidad. Los casos en los que se detectan nulos son:
  - Dimensión de Estudiantes: Se detectaron nulos masivos en la variable date_unregistration (22,521 registros), lo cual es un indicador positivo de persistencia, ya que representa a los alumnos que no han abandonado el curso. Asimismo, variables como imd_band (1,111 nulos) y date_registration (45 nulos) requieren un tratamiento diferente para evitar sesgos de selección al eliminar registros.
  - Dimensión de Evaluaciones: Se identificaron 173 registros sin calificación (score) y 2,865 sin fecha de entrega definida. Estas inconsistencias en los hitos evaluativos marcan la necesidad de un filtrado estricto en la fase de preprocesamiento para garantizar la veracidad de las etiquetas de rendimiento.
- Dimensión de Interacciones: Los campos temporales en los logs muestran nulos que deben ser normalizados para no interrumpir el cálculo de métricas secuenciales como el momentum o el engagement pesado.
- Balanceo de clases y representatividad: El análisis de la siguiente ilustración evidencia una distribución asimétrica de los resultados académicos, con una representación mayoritaria de las categorías de aprobado y abandono y una presencia reducida de sobresalientes y suspensos. Este desbalance justifica la adopción de métricas robustas frente a clases mayoritarias, como el F1-Score, así como el uso de arquitecturas de *Deep Learning* capaces de identificar patrones sutiles en grupos minoritarios.

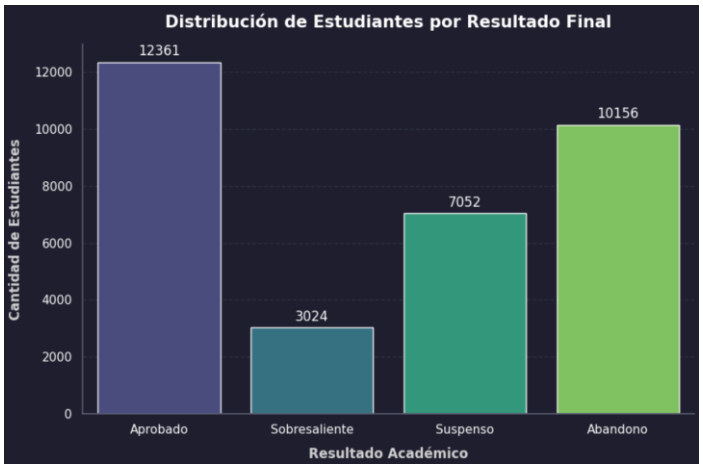

Ilustración 4 Distribución de estudiantes por resultado académico. Autoría propia - Apéndice 1 - Notebook I

- Ruido diario de interacciones diarias frente a ruido semanal: Al contrastar el promedio de interacciones, se observó que el análisis diario presenta un ruido de alta frecuencia difícil de modelar. Por el contrario, la agregación semanal ofrece una señal limpia con patrones identificables. Esta observación justifica la decisión técnica de transformar los registros brutos en secuencias semanales para mejorar la capacidad predictiva del modelo.

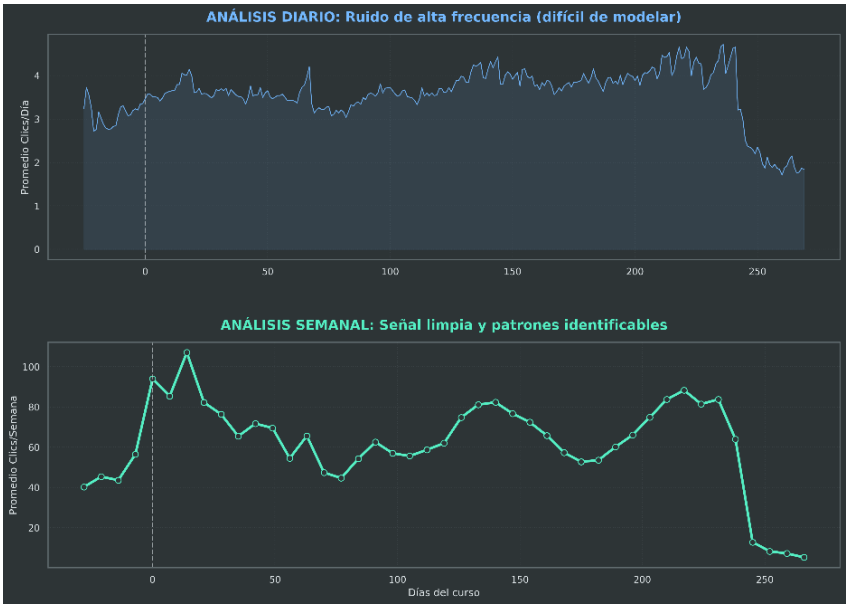

Ilustración 5 Análisis comparativo de de interacciones: Ruido diario frente a señal semanal agregada. Autoría propia - Apendice I - Notebook I

- Nivel de riqueza en el código postal del estudiante: Se observa una relación directa entre el nivel de riqueza medio del código postal del estudiante y los resultados. Vemos como la tasa de abandono y suspensos es más alta en barrios pobres, frente a una mayor tasa de aprobados y sobresalientes en barrios más adinerados.

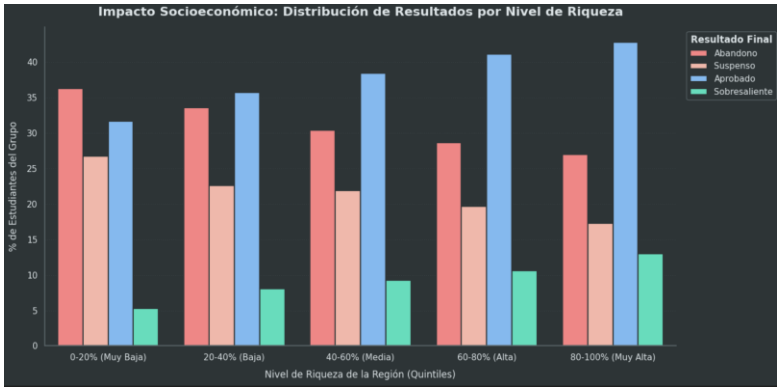

Ilustración 6 Contraste entre riqueza y resultados académicos - Autoría Propia - Apéndice I - Notebook I

- La influencia del tipo de curso en la cantidad de iteraciones: Como muestra el análisis diferencial (siguiente imagen), la intensidad de interacción varía drásticamente según la materia; por ejemplo, el curso FFF triplica la actividad del BBB. Sin un ajuste previo, el algoritmo de clustering sufriría un sesgo de volumen, agrupando a los estudiantes por la asignatura matriculada en lugar de por su comportamiento real, lo que invalidaría la identificación de patrones de compromiso genuinos.

  Para evitar esto, en lugar de contar clics totales, se mide qué tanto destaca un alumno respecto al promedio de su propia clase. Así, podemos comparar con justicia a estudiantes de distintos cursos, asegurando que el análisis se centre en el ritmo y la constancia individual, sin importar si la materia exige muchas o pocas interacciones.

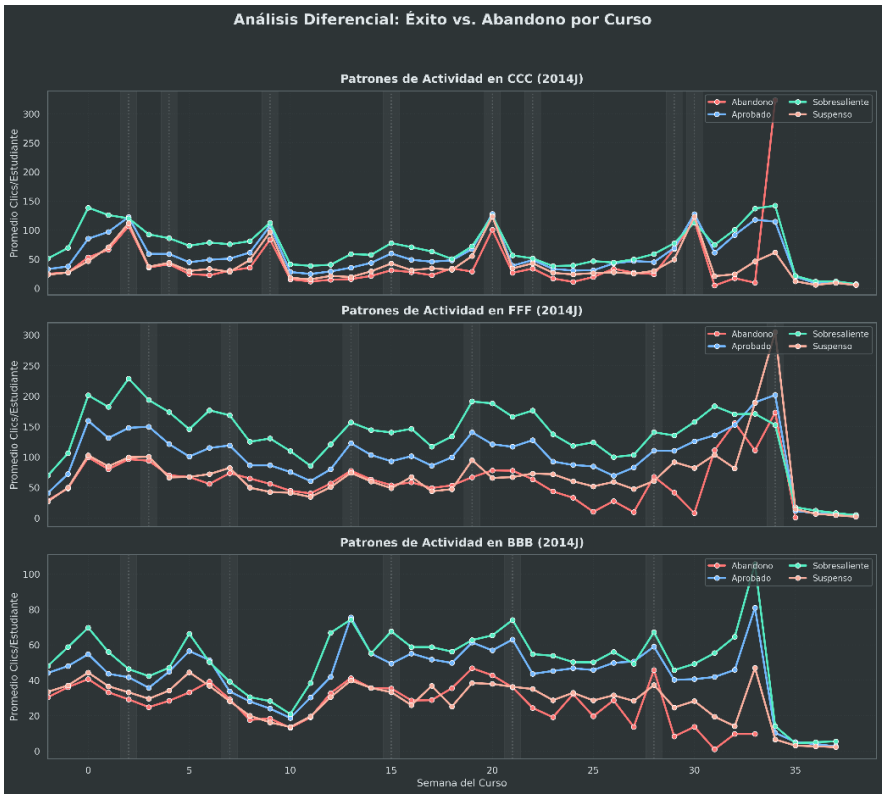

Ilustración 7 Comparación de interacciones por curso.
Autoría propia - Apéndice I - Notebook I

- Si nos fijamos en la siguiente ilustración podremos apreciar que no hay relación entre el día que se matricularon los alumnos y si se abandona o si se aprueba, por el contrario, el número de intentos previos y el número de interacciones con la plataforma educativa antes del comienzo del curso sí que están directamente relacionada con la tasa de abandono, el porcentaje de aprobados y el número de sobresalientes, por lo tanto, se las proporcionaremos a nuestro modelo:

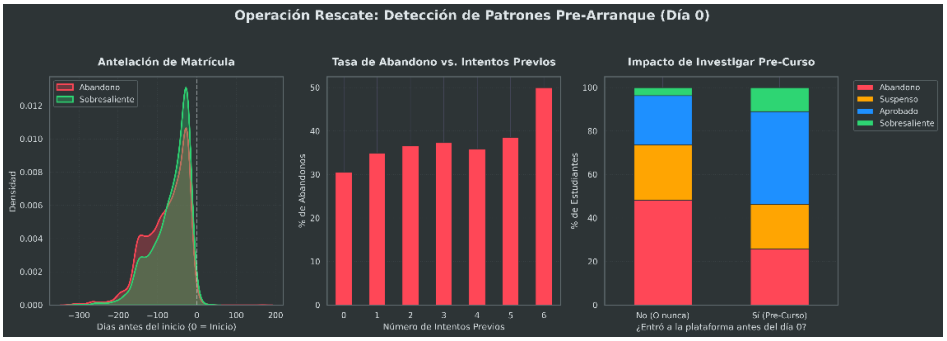

Ilustración 8 Relación entre diferentes métricas y resultados académicos.
Apéndice 1 - Notebook I

- Se puede ver también que hay una relación entre el nivel educativo previo y los resultados obtenidos. Cuanto mayor es el nivel educativo previo más probabilidades hay de que el estudiante tenga éxito.

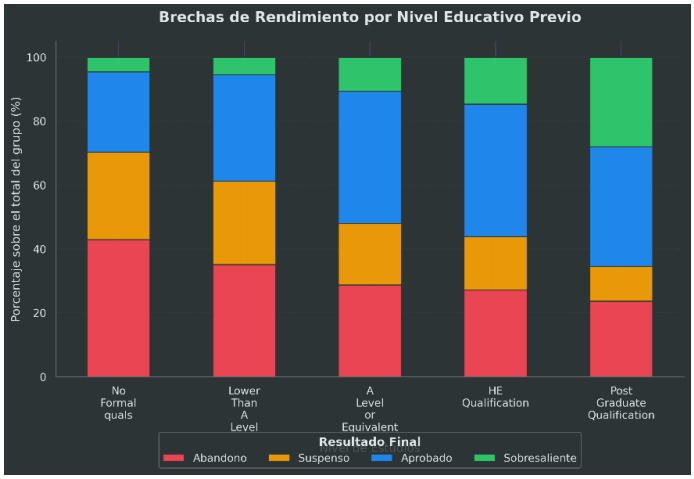

Ilustración 9 Relación entre nivel educativo y resultados académicos
Apéndice 1 – Notebook I

Es fundamental señalar que esta fase de análisis exploratorio (EDA) no constituye un bloque estático o aislado dentro del flujo de trabajo. Por el contrario, el EDA actúa como un proceso transversal y evolutivo que acompaña al ciclo de vida del proyecto. A medida que se itera en las fases de pre-procesamiento, entrenamiento de modelos y evaluación, los hallazgos obtenidos permiten volver al análisis de los datos para pulir métricas y perfeccionar la ingeniería de características.

5.3. Preprocesamiento de datos.

Una vez comprendida la naturaleza y distribución de los datos, se procedió a su preprocesamiento. En primera instancia, se llevó a cabo un tratamiento básico de la información para facilitar el Análisis Exploratorio de Datos (EDA). Posteriormente, el flujo de trabajo se bifurcó en dos enfoques metodológicos: el primero consistió en el preprocesamiento y la ingeniería de características orientados a un modelo Autoencoder, con el objetivo de aplicar técnicas de clustering sobre su espacio latente; el segundo abordó un proceso análogo, pero adaptado a la red neuronal Transformer. Por consiguiente, la presente subsección se dividirá en el preprocesamiento destinado al EDA, al Autoencoder y, finalmente, a la arquitectura Transformer."

5.3.1. Preprocesamiento para EDA.

El procesado de los datos para el análisis exploratorio de datos se llevó a cabo a partir de los datos obtenidos de la base de datos oficial de OULAD, el tratamiento de los datos se llevó a cabo en el script dataset.py dentro de la carpeta “educational_ai_analytics/0_data” (Véase apéndice I) y las salidas tras procesar los datos se genera en “data/1_interim” y en “data/2_processed”. Las acciones específicas que se llevan a cabo en este script son:

- Consolidación de los múltiples archivos tabulares del sistema (estudiantes, cursos, registros, evaluaciones e interacciones VLE) en tres grandes conjuntos de datos unificados. Durante este proceso, se estandarizaron los valores faltantes o desconocidos (originalmente representados con caracteres especiales como '?').

- Partición y protección contra fuga de datos (Data Leakage): División estratificada de la población estudiantil en conjuntos de Entrenamiento (70%), Validación (15%) y Prueba (15%).

- Durante este preprocesamiento, se prestó especial atención a la imputación de valores nulos para no distorsionar la muestra. Concretamente, las ausencias en el índice de privación múltiple (imd_band) se recategorizaron bajo la etiqueta 'Unknown' para preservar los registros sin introducir sesgos poblacionales. Por su parte, los valores faltantes en la fecha de matriculación (date_registration) se imputaron estadísticamente utilizando la mediana temporal del curso correspondiente (code_presentation), calculada estrictamente sobre el conjunto de entrenamiento para evitar fuga de datos (data leakage).

5.3.2. Preprocesamiento para Autoencoder y clustering.

La preparación de los datos para el Autoencoder se llevó a cabo en dos scripts de Python, específicamente "ae_uptow_features.py" y "day0_static_features" dentro de la carpeta "educational_ai_analytics /1_proccesed" y salida de estos scripts se generó en la carpeta "data/2_features" (Véase Apéndice I). Aunque a nivel práctico se hayan fusionado el preprocesado e ingeniería de características en un mismo script. En este epígrafe nos limitaremos a la parte de preprocesamiento.

- Para evitar la redundancia y garantizar la trazabilidad longitudinal del alumno, se construyó un identificador único compuesto por el código del estudiante, el módulo matriculado y el semestre (id_student_code_module_code_presentation). Este identificador actuó como índice principal para todas las tablas.

- Normalización condicional por curso: Para evitar que las variaciones inherentes a cada asignatura distorsionaran el análisis del comportamiento, se aplicó una normalización estadística (Z-score) sobre las variables continuas. Esta normalización no se calculó de forma global, sino de manera agrupada y específica para cada cohorte (curso y convocatoria), asegurando que el comportamiento de un estudiante se evalúe exclusivamente en relación con el promedio de sus compañeros de clase.

- Gestión de variables categóricas y valores atípicos (Demografía): Variables ordinales como el índice de privación múltiple (imd_band), el rango de edad (age_band) y el nivel educativo previo (highest_education) se mapearon a escalas numéricas continuas o discretas, facilitando su interpretación por parte de las redes neuronales.

  Adicionalmente, para la imputación de valores faltantes en estas variables categóricas, se aplicó una estrategia basada en la moda estadística del conjunto de entrenamiento.

- Tratamiento robusto de interacciones (Clipping): Dada la alta varianza en la frecuencia de interacción diaria de los estudiantes con la plataforma (clics), se implementó una técnica de clipping (acotamiento). Se limitaron los valores máximos de actividad diaria utilizando el cuantil superior (0.995) de la distribución, mitigando así el impacto de outliers extremos generados por actividad inusual (por ejemplo, bots de descarga masiva).

5.3.3. Preprocesamiento para red neuronal Transformer.

El preprocesamiento para el modelo Transformer difiere del enfoque tradicional, ya que requiere estructurar la información en secuencias temporales

en lugar de vectores planos. Este tratamiento se automatizó para transformar los registros diarios en tensores tridimensionales, ejecutando las siguientes acciones clave:

- Estructuración temporal y de identidad: Se generó un identificador único por estudiante y curso, transformando la granularidad de los registros de días relativos a índices semanales.
- Generación de tensores y máscaras: Los registros se agruparon en un tensor de dimensionalidad (N, T, F) (estudiantes, semanas transcurridas, tipos de actividad). Simultáneamente, se creó una máscara booleana (mask_activity) para indicar explícitamente al mecanismo de atención del Transformer qué semanas contienen señales de comportamiento real y cuáles son vacíos estructurales por inactividad.
- Normalización contextualizada: Para mitigar la alta varianza en la frecuencia de clics, se aplicó una transformación Log1p seguida de una estandarización (Z-score). Para prevenir la fuga de datos (data leakage), los estadísticos de normalización se calcularon estrictamente sobre el conjunto de entrenamiento, diferenciando por curso y considerando exclusivamente las semanas con actividad.

5.4. Ingeniería de características.

En el caso de la ingeniería de características tenemos dos fases: La primera consistente en la preparación y selección de los datos para el autoencoder, el cual representará la información en un espacio latente de menor dimensión que luego será clusterizado. La segunda es dependiente de la resolución de la primera fase, dado que los resultados de las agrupaciones de estudiantes serán inyectados a la entrada de la red neuronal Transformers.

5.4.1. Fase I: Características para Autoencoder.

El conjunto de características que alimenta el entrenamiento del modelo se construye a partir de los scripts ae_uptow_features.py y day0_static_features.py, alojados en la carpeta educational_analytics/ 1_features/ (véase el Apéndice I). Una vez procesados los datos provenientes del directorio data/2_processed, las variables resultantes para el autoencoder se exportan a data/3_features. La arquitectura de estos datos se fundamenta en dos bloques principales:

- Información que se tiene del estudiante desde el comienzo del curso, es decir desde el día cero:
  - Datos sociodemográficos: Con el fin de cumplir con aspectos éticos y legales, se descartó incluir información como discapacidad o género. Sin embargo, dado el poder discriminativo que se encontró en EDA de variables como Región, índice de riqueza del código postal, nivel educativo previo, número de veces que intento previamente el curso, etc. Se opto por incluirlas
  - Características de Arranque (Pre-inicio del curso): Se modeló el nivel de proactividad antes del comienzo oficial de las clases (Día 0). Para ello, se calcularon métricas como la intensidad previa (prestart_intensity), que mide la relación entre los clics totales y los días activos, y la anticipación (prestart_anticipation), que cuantifica cuántos días antes del inicio el alumno comenzó a interactuar con la plataforma.
- Información acumulada en diferentes ventanas de tiempo: El objetivo es que para diferentes ventanas de tiempo (desde la semana cero hasta la semana X) se calcules los valores acumulados de clics en los

diferentes tipos de actividad, junto con calificación obtenida hasta la fecha, etc.

- Métricas de Compromiso Ponderado (Weighted Engagement): Se descartó el uso del conteo simple de clics totales, ya que introduce un sesgo de volumen que no refleja necesariamente el aprendizaje real. En su lugar, se creó una métrica de compromiso ponderado (total_weighted_engagement) donde se asignó un mayor peso (1.3) a actividades que requieren esfuerzo cognitivo activo (como cuestionarios o herramientas colaborativas) y un peso menor (0.8) a la lectura pasiva de recursos.

- Dinámicas Temporales y Momentum: Para capturar el ritmo y la constancia del estudiante, se derivaron variables secuenciales críticas:
  - Tendencia de esfuerzo (effort_slope): Indica si la actividad del alumno va en aumento o en declive.
  - Momentum a corto plazo (momentum): Cuantifica la aceleración o desaceleración del compromiso.
  - Rachas y latencia: Se extrajeron métricas como la racha final de semanas consecutivas activas (streak_final) y la cantidad de semanas transcurridas desde la última actividad (weeks_since_last_activity), fundamentales para detectar riesgo inminente de abandono.
  - Diversidad y Dispersión del Aprendizaje: Se diseñaron indicadores para perfilar el estilo de navegación del alumno. Entre ellos destaca la entropía temporal (temporal_entropy_uptoW), que diferencia a los estudiantes constantes de aquellos que concentran su actividad en picos esporádicos. Asimismo, métricas como el top1_share (proporción de clics en la actividad principal) y el índice

de curiosidad (curiosity_index) permitieron medir si el estudiante explora transversalmente los recursos o se limita a un único tipo de material.

- Evolución del Rendimiento (Assessments): Se generaron características de evaluación que incluyen la ratio de aprobados (pass_ratio), la tendencia en las calificaciones (score_slope), la proporción de entregas tardías (late_ratio) y el tiempo transcurrido desde la última entrega (weeks_since_last_submission). Adicionalmente, se formuló un índice compuesto (api_index) que pondera la nota media lograda por el logaritmo del volumen de entregas realizadas.

La concatenación de todas estas 58 características conformó el vector de alta dimensionalidad que alimentó la arquitectura del Autoencoder. Esto permitió a la red comprimir la información en un espacio latente, extrayendo patrones de comportamiento no lineales, limpios de ruido y altamente accionables desde la perspectiva pedagógica. Para una revisión exhaustiva del conjunto de datos resultante, el Apéndice II detalla las 58 características definitivas con las que se entrenó el modelo.

5.4.2. Fase II: Características para Transformers.

La segunda fase de la ingeniería de características tiene como propósito construir el conjunto de datos que alimentará el modelo secuencial. A diferencia del Autoencoder, que evalúa vectores estáticos de datos agregados, la red neuronal Transformer requiere una representación dinámica enriquecida con el contexto global del estudiante.

Esta fase de implemento en el script transformer_GLU_classifier.py dentro de la carpeta educational_ai_analytics/modeling/transformes, la exploración

previa para modelar este script se encuentra en el notebook 5 (Apéndice I). La arquitectura de las características generadas en este script se estructura en los siguientes tres bloques fundamentales:

- Matriz Secuencial de Interacciones (Comportamiento Temporal): Constituye el núcleo dinámico del modelo. En lugar de métricas acumuladas, la información se estructura en un tensor tridimensional de dimensionalidad (N x T x F) (estudiantes, semanas transcurridas, tipos de actividad). Esto permite a la red evaluar la evolución pura de las interacciones semanales del alumno con cada recurso del LMS (foros, cuestionarios, lecturas) de forma cronológica.
- Representaciones Latentes y Arquetipos (Soft Labels): En este punto se materializa la arquitectura híbrida del proyecto. A la secuencia temporal se le inyectan las probabilidades continuas de pertenencia de cada estudiante a cada uno de los clústeres generados previamente por el Modelo de Mixtura de Gaussianas (GMM). Alimentar al Transformer con esta distribución de probabilidades (soft labels), en lugar de una clase discreta, proporciona al modelo un contexto rico sobre el arquetipo de aprendizaje del estudiante, estabilizando su optimización. También se incorporan métricas complejas derivadas en la Fase I, como la entropía temporal.
- Características Estáticas y Perfil de Arranque (Día 0): Para contextualizar el punto de partida del estudiante, se concatena al tensor la información disponible antes de iniciar el curso. Esto incluye las variables sociodemográficas de alto poder predictivo (como el índice de riqueza del código postal, el nivel educativo previo y los intentos anteriores) junto con las métricas de proactividad previas al arranque.
- Variables acumulativas en la ventana de tiempo: Como resultado de la fase de experimentación, se optó por incorporar también las

características históricas acumuladas por el estudiante hasta la semana objeto de predicción (W). La inyección de estas métricas agregadas —calculadas inicialmente para el Autoencoder— complementa el análisis secuencial del mecanismo de atención, lo que se tradujo empíricamente en una mejora sustancial de las métricas de rendimiento del modelo final.

La integración conjunta de estos tres bloques permite al Transformer no solo procesar "qué" hace el estudiante semana a semana mediante su mecanismo de autoatención, sino contextualizarlo bajo "quién" es y a qué perfil de comportamiento general pertenece, maximizando así la precisión en la detección temprana del riesgo.

5.5. Modelado

La fase de modelado constituye el núcleo analítico y predictivo de esta investigación. Para abordar la naturaleza multidimensional del comportamiento estudiantil, se ha diseñado una arquitectura híbrida secuencial. Esta aproximación se divide en dos grandes etapas operativas: una primera fase de aprendizaje no supervisado (mediante Autoencoders y clustering) orientada a la reducción de dimensionalidad y el descubrimiento de arquetipos latentes; y una segunda fase de aprendizaje supervisado (basada en redes neuronales Transformer) especializada en modelar la evolución temporal del alumno para predecir su riesgo académico.

A continuación, se detallan los fundamentos técnicos, la experimentación y las decisiones arquitectónicas que sustentan cada uno de estos modelos.

5.5.1. Autoencoder.

El modelado del Autoencoder(AE) se materializa en los scripts autoencoder.py donde se construye la arquitectura del AE con Keras y

train_autoencoder.py donde se especifica la manera en la que se entrena, todos estos ficheros se encuentran en la carpeta "educational_ai_analytics/2_modeling/ae" (véase apéndice I).

La estrategia principal consiste en entrenar un único autoencoder global que procesa simultáneamente todas las ventanas semanales (5, 10, 15, 20 y 25). Esta decisión se fundamenta en tres motivos técnicos: primero, garantiza la coherencia del espacio latente, creando un marco de referencia universal que permite rastrear la evolución del estudiante sin las incompatibilidades de usar modelos independientes. Segundo, actúa como una técnica implícita de aumento de datos (*data augmentation*), multiplicando las muestras disponibles y reduciendo el sobreajuste (*overfitting*). Finalmente, fomenta la extracción de arquetipos universales, obligando a la red a identificar patrones de interacción como el *momentum* independientemente de la semana en la que ocurran, delegando el análisis estrictamente secuencial al modelo Transformer.

Adicionalmente, la arquitectura implementada se enmarca en la categoría de redes Deep Clustering Networks (DCN), inspirada en los principios del Deep Embedded Clustering (DEC). A nivel de optimización, esto significa que el modelo no se limita únicamente a la reducción de dimensionalidad, sino que es guiado por una función de pérdida dual durante su fase de entrenamiento.

Por un lado, la red minimiza una pérdida de reconstrucción (empleando la métrica Huber Loss) para garantizar que el espacio latente retenga fielmente la información estructural y las características originales del estudiante. Por otro lado, incorpora simultáneamente una pérdida de agrupamiento basada en la divergencia de Kullback-Leibler (KL). Mediante esta optimización conjunta, el autoencoder genera embeddings que no solo están limpios de ruido, sino que

están matemáticamente forzados a ser altamente discriminativos para la segmentación de perfiles de aprendizaje.

Para llevar a cabo esta optimización dual de manera estable, el proceso de entrenamiento del modelo (implementado en el script train_autoencoder.py) no se ejecuta en un solo paso, sino que se estructura cuidadosamente en tres fases secuenciales:

- Fase 1: Pre-entrenamiento (Reconstrucción): En esta etapa inicial, la red se entrena de manera estándar como un autoencoder tradicional, desactivando temporalmente el objetivo de clustering. El objetivo exclusivo es minimizar el error de reconstrucción mediante la métrica Huber Loss. Esto garantiza que el modelo aprenda primero a proyectar los datos en un espacio latente que retenga fielmente toda la información estructural (estática y dinámica) de los estudiantes, construyendo una base sólida y libre de ruido.
- Fase 2: Inicialización de Centroides (K-Means): Una vez obtenido un espacio latente pre-entrenado y estable, se extraen los embeddings de los estudiantes y se aplica sobre ellos el algoritmo K-Means clásico. Los centroides encontrados (k=X, según lo determinado en la experimentación) no se usan como resultado final, sino que se inyectan en la red neuronal para inicializar los pesos de la capa específica de clustering. Este paso es un requisito clave del enfoque DEC, ya que proporciona a la red un punto de partida óptimo antes de aplicar la pérdida probabilística.
- Fase 3: Optimización Conjunta (Entrenamiento DCN): Es la fase central y más compleja del modelado. Aquí, la red se afina (fine-tuning) minimizando simultáneamente la pérdida de reconstrucción y la divergencia de Kullback-Leibler (KL). Durante esta etapa, el modelo calcula una distribución objetivo (P) matemática que fuerza a las predicciones probabilísticas a ser más "nítidas" (acercando los estudiantes

a su centroide más próximo y alejándolos del resto). Esta distribución se recalcula iterativamente a lo largo de las épocas, guiando al espacio latente para que aumente la cohesión intra-clúster sin perder la capacidad de reconstrucción original.

Adicionalmente, a nivel de ingeniería de Machine Learning, el bucle de entrenamiento incorpora técnicas avanzadas para garantizar la robustez del modelo, tales como Early Stopping basado en la pérdida conjunta para evitar el sobreajuste (overfitting), y el uso de precisión mixta (mixed_float16) para optimizar drásticamente los tiempos de cómputo en la GPU.

5.5.2. Embeddings

Los embeddings —entendidos como representaciones comprimidas de los vectores de datos de los estudiantes en un espacio latente de menor dimensionalidad— fueron creados empleando el autoencoder descrito en el epígrafe anterior. De forma complementaria, y con el objetivo de disponer de una línea base (baseline) interpretable, se entrenó también un modelo PCA, ajustando su número de componentes al mismo tamaño dimensional del espacio latente del autoencoder. Esto nos permite reducir la dimensionalidad de los datos siguiendo dos filosofías diferentes (PCA y AE). Esto permite realizar un comparación que evaluará cuál de las dos técnicas resulta más adecuada como etapa previa al clustering.

Para la generación de los embeddings se utilizó el script "generate_embeddings.py" en la carpeta educational_ai_analytics/2_modeling /ae (véase apéndice I), este script toma como entrada los datos en la carpeta "data/3_features" y a partir de ella se generan los embeddings tanto para AE (ae_latent.csv) como para PCA(pca_latent.csv), estos vectores característicos

de los estudiantes en el espacio latente son los que se utilizarán en la siguiente fase de Clustering.

5.5.3. Clustering

El modelado de clustering fue un proceso iterativo del cual destacan dos ajustes metodológicos clave para lograr un sistema de alerta temprana robusto:

- Prevención de fuga de datos: Se pasó de una ventana temporal única a ventanas acumulativas semanales, garantizando que el modelo agrupe a los estudiantes basándose estrictamente en la información histórica disponible hasta ese momento.
- Mejora del espacio latente: Se incorporó una pérdida adicional basada en la divergencia de Kullback–Leibler (KL) al Autoencoder, lo que generó representaciones (embeddings) mucho más cohesivas y separables que minimizando únicamente el error de reconstrucción.

El detalle completo de los experimentos para tomar decisiones de clustering puede consultarse en el Notebook 4 dentro de la carpeta Notebooks del repositorio (véase Apéndice I).

En dicho notebook se exploran dos familias de combinaciones: (PCA o AE) + K-Means y (PCA o AE) + Gaussian Mixture Model (GMM). No obstante, para evitar extender en exceso la memoria, en este epígrafe se analiza únicamente la segunda combinación, es decir, PCA o AE con GMM. Esta decisión se alinea con lo expuesto y justificado en el epígrafe anterior: por un lado, la naturaleza del problema no sugiere fronteras "duras" entre perfiles de estudiantes, y por otro, el uso posterior de modelos basados en Transformers se beneficia de representaciones probabilísticas (p. ej., pertenencias blandas o distribuciones), lo que hace al GMM más coherente con el enfoque global.

Para evaluar la calidad de las agrupaciones formadas, nos apoyamos en dos métricas: el Criterio de Información Bayesiano (BIC) y el Criterio de Akaike (AIC). En términos sencillos, ambos indicadores buscan el equilibrio perfecto: evalúan qué tan bien logra el modelo representar la realidad de los datos, pero le aplican una penalización matemática si utiliza una cantidad innecesaria de grupos para lograrlo. Por lo tanto, un valor más bajo indica siempre un modelo mejor y más eficiente. Sin embargo, no siempre se escoge el valor más bajo, sino que aquel que da equilibrio entre un valor bajo de BIC/AIC y utilidad interpretativa.

En la figura siguiente se muestra la evolución de los criterios de información BIC/AIC para distintos valores de k (número de componentes) al aplicar GMM sobre embeddings generados mediante PCA y mediante autoencoder.

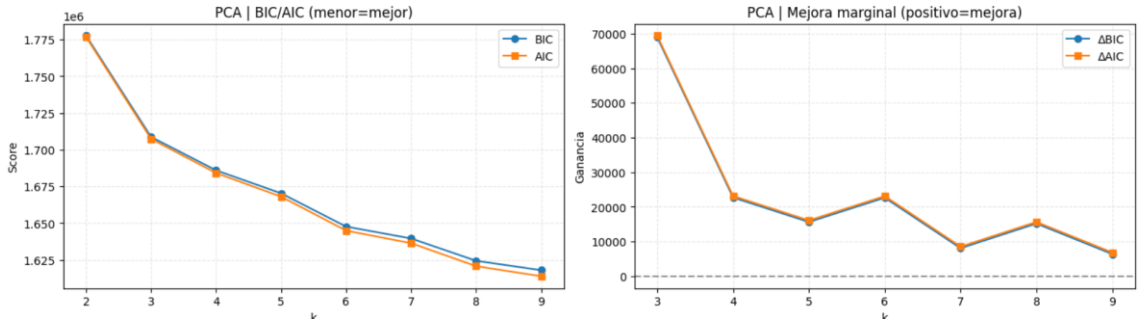

Ilustración 10 Comparación BIC/AIC en PCA
Autoría propia - Apéndice I - Notebook 4

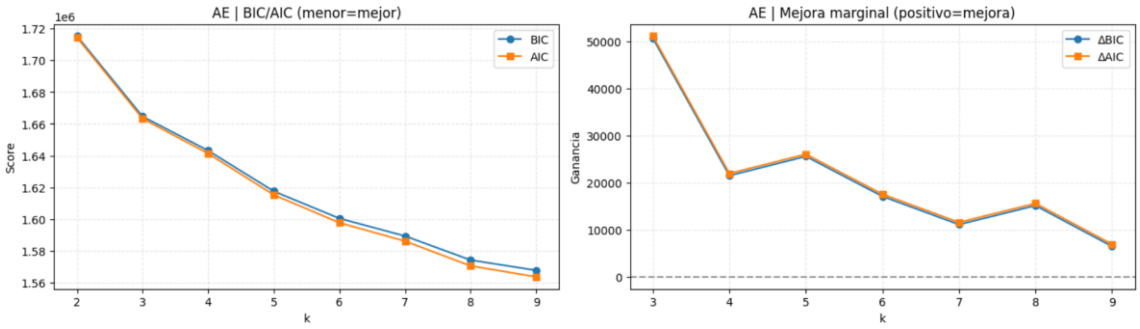

Ilustración 11 Evolución BIC/AIC con AE -
Autoría propia - Apéndice I - Notebook 4

Al observar las gráficas, la primera diferencia notable radica en el ajuste general de los modelos. Aunque ambos enfoques sitúan sus métricas en el mismo orden de magnitud (entre 1.56 y 1.77 millones), la Ilustración 13 (AE) presenta valores de BIC y AIC consistentemente inferiores a los de la Ilustración 12 (PCA) para cualquier número de componentes. Esta diferencia constante evidencia que las representaciones latentes extraídas por el Autoencoder logran un ajuste matemático superior frente a la reducción lineal tradicional.

Además de la superioridad en los valores absolutos, la evolución de las ganancias a medida que aumenta k revela dinámicas distintas. A diferencia de lo que podría esperarse, ninguna de las dos técnicas registra empeoramientos (valores negativos) en su gráfica de mejora marginal dentro de este rango; es decir, añadir clústeres siempre aporta alguna ganancia matemática. Sin embargo, en el caso de PCA, la curva marginal presenta fluctuaciones más inestables (con descensos pronunciados en k=5 y k=7, y un pico anómalo en k=6). Esta volatilidad sugiere que la reducción de dimensionalidad lineal no logra desenredar adecuadamente la complejidad de los datos, dificultando que el algoritmo GMM encuentre fronteras naturales constantes.

Por el contrario, la gráfica de mejora marginal para el Autoencoder muestra una estructura de "escalones" mucho más lógica para definir perfiles. Se observa un claro repunte de ganancia al agrupar en k=5, seguido de un descenso suave y un último máximo local en k=8. Sin embargo, al probar con k=9, la mejora marginal se desploma de forma drástica acercándose a la línea del cero. Esto señala un claro punto de inflexión: añadir un noveno clúster incrementa la complejidad del modelo sin aportar prácticamente ningún beneficio que justifique dicho agrupamiento (riesgo de sobreajuste).

A la luz de estos resultados empíricos, se extraen dos conclusiones fundamentales para la definición de la arquitectura final del sistema:

- Superioridad del Autoencoder para la extracción de características: Las representaciones no lineales generadas por el AE (optimizadas adicionalmente con la pérdida de clustering) permiten al algoritmo GMM encontrar una estructura de agrupamiento mucho más definida, robusta y con un ajuste matemáticamente superior al del PCA.
- Selección del número óptimo de clústeres (k=5): El valor k=5 emerge como la configuración óptima para el modelo GMM sobre los embeddings del AE. Si bien la matemática pura podría sugerir particiones mayores, k=5 representa el punto de máximo beneficio en el compromiso entre el ajuste técnico y la parsimonia pedagógica. Dado que el problema subyacente evalúa cuatro resultados académicos base (Aprobado, Sobresaliente, Suspenso y Abandono), segmentar a la población en cinco clústeres permite mapear de forma natural estos arquetipos de comportamiento principal —añadiendo flexibilidad para un perfil transicional— sin fragmentar la información en micro-grupos que perderían interpretabilidad para el docente.

Por consiguiente, para la generación de las secuencias de entrada que alimentarán posteriormente al modelo basado en Transformers, se consolida el uso de un modelo de mezclas gaussianas (GMM) de 5 componentes entrenado sobre el espacio latente del Autoencoder. Esta configuración permite perfilar a los estudiantes en cinco estados probabilísticos distintos cada semana, proporcionando una granularidad rica, interpretable y matemáticamente fundamentada para la detección temprana de riesgo.

Para dotar de mayor robustez a la selección del modelo, de forma complementaria al análisis de componentes, se ejecutó una búsqueda exhaustiva de hiperparámetros (Grid Search) para el algoritmo de Mixtura Gaussiana (GMM) resultando en los datos aportados en la siguiente imagen.

	Config			Silueta	Calinski-Harabasz	Davies-Bouldin	BIC	AIC	Log-Likelihood
	Embedding	Covarianza	Init	↑ MAXIMIZAR	↑ MAXIMIZAR	↓ MINIMIZAR	↓ MINIMIZAR	↓ MINIMIZAR	↑ MAXIMIZAR
0	AE	diag	random	0.1528	2027.1	3.0920	1618229.7	1615948.0	-35.4483
1	AE	spherical	random	0.1468	1768.7	3.4152	1645506.0	1644309.0	-36.0766
2	AE	spherical	k-means++	0.1440	2098.1	2.8805	1644929.6	1643732.5	-36.0640
3	AE	tied	kmeans	0.1219	2283.3	2.4023	1564958.7	1560540.0	-34.2208
4	AE	spherical	kmeans	0.1204	2265.1	2.5843	1645179.6	1643982.5	-36.0694
5	AE	diag	k-means++	0.1190	2265.9	2.5882	1617506.9	1615225.2	-35.4325
6	AE	diag	kmeans	0.1187	2266.2	2.5883	1617513.1	1615231.5	-35.4326
7	AE	full	kmeans	0.0472	1778.9	3.9701	1325372.4	1307906.8	-28.6056
8	PCA	tied	kmeans	0.0465	779.1	2.9984	1689184.8	1684766.2	-36.9468
9	AE	full	random	0.0409	1543.8	3.8240	1271243.6	1253778.0	-27.4178
10	PCA	tied	k-means++	0.0342	398.3	4.1681	1803996.0	1799577.4	-39.4663

Ilustración 12 Comparativa de resultados en diferentes métricas entre AE y PCA

Este proceso evaluó iterativamente diversas configuraciones relativas al tipo de matriz de covarianza (diag, spherical, tied, full) y a la estrategia de inicialización de los centroides (random, k-means, k-means++), aplicando este barrido tanto sobre las representaciones latentes generadas por el Autoencoder (AE) como por el PCA.

Los resultados recogidos en la siguiente ilustración exponen las mejores configuraciones obtenidas tras la experimentación. Como se puede apreciar con claridad, las representaciones basadas en el Autoencoder monopolizan las primeras posiciones del ranking global. Mientras que las configuraciones con AE logran maximizar las métricas de cohesión como la Silueta (alcanzando 0.1528) y minimizar el solapamiento (Davies-Bouldin), las alternativas basadas en PCA quedan relegadas a los últimos lugares del Top 10, presentando un rendimiento estructural y probabilístico notablemente inferior. Esta evidencia empírica consolida definitivamente la elección de la arquitectura no lineal para la fase de agrupación.

5.5.4. Transformer I: Modelado.

Para el modelado de la red neuronal Transformers se uso Keras, el archivo en el que se materializa la implementación del modelo es "transformer_GLU_classifier.py" dentro de la carpeta "ai_educational_analytics/2_modeling/transformer", y su entrenamiento se realiza en el archivo "train_transformer.py" (Apéndice II), complementariamente hay muchos scripts que sirven de utilidades (finetuning, comparaciones de métricas entre entrenamientos, etc.)

Esta arquitectura se concibió con un enfoque "multimodal": separa la información en dos grandes ramas paralelas para procesarla de la manera más adecuada antes de tomar una decisión conjunta.

- Procesamiento en paralelo: El modelo recibe la información dividida en dos naturalezas muy distintas, y utiliza una rama especializada para cada una con el objetivo de traducirlas a un espacio latente común:
  - Rama de secuencias temporales: Toma las interacciones del estudiante semana a semana (clics, entregas, tiempo de conexión). Esta información pasa por una capa de proyección que transforma estas variables brutas en representaciones vectoriales (embeddings) más compactas y ricas en significado.
  - Rama de contexto estático: Paralelamente, todas las variables estáticas (datos demográficos, actividad previa al inicio del curso, métricas acumuladas y el perfil probabilístico asignado por el Autoencoder/GMM) se procesan a través de una red neuronal densa clásica. El objetivo de esta rama es destilar el "perfil base" del estudiante y proyectarlo a la misma dimensión que los datos temporales.

- Fusión temprana (Early Fusion): Mediante un mecanismo de compuertas (Gated Fusion), el modelo inyecta el "perfil base" del estudiante en cada una de las semanas de su secuencia temporal. De este modo, la red aprende a interpretar una misma acción (por ejemplo, una bajada de actividad en la semana 3) de forma distinta dependiendo de si el alumno tiene un perfil histórico de alto riesgo o si es un estudiante excelente que simplemente tuvo una semana atípica.

- El motor de atención: La secuencia, ya enriquecida con el contexto estático del alumno, entra en el corazón del modelo: el bloque Transformer. Aquí, los mecanismos de "Atención Multicabezal" (Multi-Head Attention) analizan la secuencia completa de golpe. La red aprende a conectar eventos pasados con comportamientos recientes, entendiendo la evolución del estudiante. Para que este análisis sea preciso, se utilizan máscaras lógicas; estas indican al modelo qué semanas no tuvieron actividad real o cuáles son simplemente relleno (padding), permitiendo que la red ignore el "ruido" y se concentre solo en la información válida.

- Decisión final y fusión tardía (Late Fusion): Tras el análisis secuencial, el Transformer comprime toda la evolución del estudiante en un único vector de resumen. Para tomar la decisión final con la máxima garantía, el modelo junta tres piezas clave de información:

    1. El resumen de la evolución temporal (lo que acaba de calcular el Transformer).

    2. El perfil estático procesado.

    3. Los datos estáticos crudos originales (un "bypass" directo para que el clasificador no olvide datos cruciales como la demografía exacta).

Finalmente, esta información combinada pasa por un último bloque de red neuronal (el Head de clasificación) que emite la predicción probabilística (Aprobado / Suspenso) sobre el estado final del estudiante en el curso.

5.5.5. Transformer II: Estrategia del entrenamiento y configuración de clases.

Una vez diseñada la arquitectura multimodal, el proceso de aprendizaje del modelo (gestionado mediante el script train_transformer.py) se configuró estratégicamente para afrontar el fuerte desbalanceo natural de los alumnos y la necesidad de adaptar el sistema a diferentes definiciones de "riesgo académico".

Para lograr que el modelo sea útil en un entorno real, la estrategia de entrenamiento se fundamenta en los siguientes pilares:

- Flexibilidad en los escenarios de predicción: El sistema no está anclado a un único objetivo. Permite resolver el problema con distintos niveles de granularidad: desde una clasificación binaria directa (estudiantes en riesgo frente a casos de éxito, permitiendo aislar abandonos o suspensos según convenga), pasando por una visión trinaria (fracaso, abandono y éxito), hasta una clasificación cuaternaria que respeta todas las calificaciones originales (incluyendo las matrículas de honor).
- Aprendizaje enfocado en la minoría (Focal Loss): En un curso habitual, la gran mayoría de los alumnos aprueba y solo una minoría abandona. Si el algoritmo se entrenara de forma tradicional, tendería a ignorar a los alumnos en riesgo porque le resultaría más "rentable" predecir siempre que todos van a aprobar. Para neutralizar este sesgo, se emplea una técnica matemática llamada Focal Loss. Este mecanismo penaliza severamente a la red cuando se equivoca en los casos raros (la minoría en riesgo) y le resta importancia a los alumnos que son evidentes,

obligando al modelo a concentrarse en los estudiantes más difíciles de detectar.

- Ajuste dinámico de la alerta (Threshold Tuning): En los escenarios de predicción de riesgo, disparar una alerta simplemente porque la probabilidad cruza la barrera del 50% rara vez es la mejor decisión. El sistema incorpora un algoritmo que evalúa cientos de puntos de corte de forma automática durante la fase de validación. Su objetivo es encontrar el umbral matemático óptimo que logre el equilibrio perfecto: maximizar la cantidad de alumnos en peligro detectados (exhaustividad), garantizando al mismo tiempo que no se sature a los docentes con falsas alarmas (precisión).
- Estabilidad y prevención de sobreajuste: Para asegurar que el modelo aprenda a deducir patrones reales y no se limite a memorizar el comportamiento de los alumnos de entrenamiento, el sistema monitoriza su propio rendimiento constantemente. Si detecta que el aprendizaje se estanca, reduce de forma autónoma la velocidad a la que actualiza sus conocimientos para realizar ajustes más finos. Si la mejora se detiene por completo, el entrenamiento finaliza prematuramente (Early Stopping) y rescata la versión exacta de la red que logró el mejor equilibrio predictivo.

Finalmente, para gobernar la enorme complejidad de este ecosistema predictivo y garantizar la robustez de los resultados, el proyecto se dotó de una infraestructura de experimentación automatizada. Conscientes de que el rendimiento de una red Transformer es altamente sensible a su configuración, se desarrolló un script de búsqueda diseñado para evaluar miles de combinaciones de hiperparámetros (tasas de aprendizaje, dimensiones latentes, número de cabezas de atención, ratios de dropout, etc.). Para gestionar la ingente cantidad de modelos generados en este proceso, se implementó paralelamente un sistema de evaluación comparativa automática. Esta herramienta rastrea, archiva y

contrasta el rendimiento de cada iteración, permitiendo auditar de un vistazo qué modificaciones se traducen en mejoras reales (aumentos en Recall o precisión) y cuáles provocan un empeoramiento.

Gracias a este marco de trabajo, el ajuste fino del modelo dejó de ser un proceso de ensayo y error manual para convertirse en una toma de decisiones guiada estrictamente por la evidencia empírica.

5.6. Evaluación

A nivel técnico, el proceso de evaluación se ha diseñado de manera transversal a lo largo de los diferentes scripts y cuadernos de experimentación del proyecto. Para garantizar una trazabilidad rigurosa, todos los reportes, gráficas y métricas generadas durante las fases de validación se exportan automáticamente al directorio reports/, mientras que los archivos con los pesos de los modelos finalmente entrenados se preservan en la carpeta models/ (Véase apéndice I).

El presente epígrafe tiene como propósito establecer el marco teórico y justificar la elección de las métricas que rigen esta evaluación, reservando el análisis empírico de las cifras obtenidas para el capítulo de Resultados.

Adicionalmente, la evaluación de un sistema predictivo en el ámbito de la Minería de Datos Educativos no puede limitarse a cuantificar su tasa de acierto global. Dado que el objetivo primordial del proyecto es construir una alerta temprana para permitir la intervención docente, el modelo debe ser evaluado bajo una óptica multidimensional: su robustez estadística frente a datos desbalanceados, el análisis del coste del error y su interpretabilidad pedagógica.

5.6.1. Evaluación de Autoencoder.

Dado que el ecosistema propuesto incluye un bloque de aprendizaje no supervisado, su éxito no puede medirse con métricas de clasificación tradicionales. La calidad del Autoencoder y del agrupamiento posterior se evaluará mediante dos enfoques:

- Fidelidad de Reconstrucción: Monitorización de la pérdida de reconstrucción (Huber Loss) durante el entrenamiento, garantizando que la compresión al espacio latente no destruya la información estructural del estudiante.
- Clustering como tarea derivada: La prueba definitiva de la utilidad del Autoencoder no es su capacidad de compresión aislada, sino su impacto empírico en la fase de agrupamiento. Se contrastará si el espacio latente generado por el AE permite encontrar perfiles de estudiantes más definidos y robustos frente al uso de reducciones lineales clásicas (PCA).

5.6.2. Evaluación de Clustering.

Para medir la calidad de las agrupaciones generadas, la evaluación no se limita a un único indicador, sino que aborda el problema desde dos dimensiones complementarias: la forma geométrica de los grupos y su eficiencia matemática.

- Calidad Estructural de la Segmentación: Esta dimensión evalúa visual y geométricamente cómo se distribuyen los estudiantes en el espacio latente. Para comprobar si los grupos están bien construidos, utilizamos tres métricas:
  - Coeficiente de Silueta: Mide qué tan "seguro" está un estudiante en su grupo. Matemáticamente, evalúa si los alumnos de un mismo clúster están muy juntos entre sí (cohesión) y, al mismo tiempo, muy

lejos de los alumnos de otros perfiles (separación). Cuanto más alto es el valor, más claras son las fronteras del grupo.

- Índice de Calinski-Harabasz: Actúa como un medidor de densidad. Premia matemáticamente a los modelos que logran formar grupos que parecen "islas": muy compactos por dentro y muy alejados del resto de grupos. Un valor alto indica una estructuración fuerte y bien definida.
- Índice de Davies-Bouldin: Funciona como un detector de solapamiento. Su objetivo es calcular cuánto se mezclan o se "pisan" las fronteras de los diferentes clústeres. Al revés que los anteriores, aquí se busca el valor más bajo posible, ya que penaliza a los grupos que se confunden entre sí.

- Criterio de Información Bayesiano (BIC) y el Criterio de Akaike (AIC). En términos prácticos, estas dos métricas buscan el equilibrio entre precisión y simplicidad. Miden con qué exactitud el modelo representa la realidad de los estudiantes, pero le aplican una "multa" o penalización matemática por cada grupo extra que se crea de forma innecesaria. De esta manera, nos garantizan encontrar el modelo más eficiente: aquel que explica bien los datos utilizando la menor cantidad de clústeres posible (por lo que siempre buscaremos los valores más bajos en ambos criterios).

5.6.3. Evaluación del Transformer.

A diferencia de los modelos tradicionales donde el éxito se mide habitualmente por la exactitud global (Accuracy), la evaluación de la red neuronal Transformer debe alinearse estrictamente con su propósito pedagógico: funcionar como un sistema de alerta temprana.

En un entorno educativo real, existe un fuerte desbalance natural en las aulas (donde la gran mayoría del alumnado aprueba y solo una minoría fracasa o abandona). En este contexto, el coste de cometer un error no es simétrico. Para un docente, es mucho más grave no detectar a un alumno en riesgo de abandono (falso negativo) que enviarle un mensaje de apoyo preventivo a un alumno que finalmente iba a aprobar (falso positivo). Por consiguiente, el desempeño del modelo se evaluará bajo el siguiente conjunto de métricas especializadas:

- Recall (Exhaustividad): Constituye la métrica principal para un sistema de alerta. Mide la proporción de estudiantes en riesgo real que el modelo logra identificar correctamente. Un Recall alto garantiza que la "red de arrastre" del sistema no deje a ningún alumno vulnerable atrás.

- Precision (Precisión): Evalúa la fiabilidad de las alertas generadas. Responde a la pregunta: de todos los estudiantes que el modelo marcó en riesgo, ¿cuántos lo estaban realmente? Actúa como contrapeso del Recall para garantizar que no se sature al equipo docente con falsas alarmas.

- F1-Score y Exactitud Balanceada (Balanced Accuracy): Mientras que la exactitud estándar puede verse sesgada por la clase mayoritaria (los aprobados), estas métricas son robustas ante el desbalanceo. El F1-Score proporciona la media armónica entre Precisión y Exhaustividad, mientras que la Exactitud Balanceada calcula el acierto promedio ponderado de todas las clases, garantizando que el modelo es justo y aprende de los grupos minoritarios.

- AUC-OVR (Área bajo la curva - One vs Rest): Permite evaluar la capacidad del modelo probabilístico para separar y distinguir correctamente a los alumnos de las diferentes categorías (Éxito, Fracaso, Abandono) sin depender de un umbral de decisión fijo.

- Loss (Pérdida Matemática y Calibración de Pesos): Representa el error interno que la red intenta minimizar durante el entrenamiento. Es en este punto donde reside una de las características más potentes y adaptables del sistema: el modelo permite configurar la "fuerza" o el peso que se le otorga a cada clase. Dependiendo de la naturaleza del problema que se quiera resolver o de la política del centro educativo, el Transformer puede calibrarse matemáticamente para ser extremadamente sensible (penalizando fuertemente si se le escapa un abandono) o más equilibrado.

- Accuracy (Exactitud): Aunque se registrará el porcentaje de acierto global del sistema para tener una visión general, se considerará una métrica secundaria, asumiendo que un ligero descenso en la misma es aceptable si se traduce en una mejora en la detección de los casos de riesgo (Recall).

La combinación de estos indicadores permitirá realizar un análisis exhaustivo en el capítulo de Resultados, garantizando que el modelo no solo es matemáticamente sólido y configurable, sino pedagógicamente útil y seguro para su despliegue en el aula.

5.7. Dockerización y Pipelines Automatizados

Para garantizar que la arquitectura de Deep Learning sea agnóstica al hardware de diferentes dispositivos y así garantizar su reproducibilidad se ha diseñado el proyecto bajo una infraestructura modular y reproducible basada en dos pilares:

Contenerización del entorno (Docker): Se ha implementado una arquitectura de contenedores que aísla las dependencias críticas (como las bibliotecas CUDA/cuDNN para aceleración por GPU):

- Dockerfile: Define una imagen base inmutable con la versión exacta de python 3.11. Incluye todas las capas de sistema necesarias para la aceleración por hardware y las bibliotecas de cálculo numérico (TensorFlow, Scikit-learn), garantizando que el modelo se comporte igual en un servidor local que en la nube, o en cualquier hardware.
- Docker-Compose: Actúa como el orquestador del entorno, configurando de forma automática el montaje de volúmenes de datos, la gestión de puertos y la inyección de variables de entorno, permitiendo levantar el "laboratorio experimental" completo con un solo comando.
- Dev Containers: Se ha integrado un estándar de desarrollo que permite vincular el IDE directamente con el contenedor. Esto asegura que el código escrito y el código ejecutado compartan siempre el mismo ecosistema, librerías y versiones de compiladores.

Orquestación del Pipeline (Makefile): A diferencia del enfoque tradicional basado exclusivamente en notebooks experimentales, este proyecto implementa una Pipeline-as-Code profesional. Se ha diseñado un archivo maestro de orquestación (Makefile) que automatiza el flujo de trabajo completo:

- Ingesta de datos (make data): Realiza la limpieza, el limpiado preventivo de directorios y la división del dataset (Split) de forma determinista.
- Entrenamiento y Latencia (make prep_latent): Encadena de forma secuencial el entrenamiento del Autoencoder y la generación de embeddings, asegurando la integridad referencial entre los modelos y sus representaciones vectoriales.
- Reproducibilidad Total: Este sistema permite que cualquier investigador pueda replicar el experimento completo desde cero

ejecutando un comando único, eliminando la intervención manual y los posibles sesgos humanos en el preprocesamiento de los datos.

6. Resultados.

En este capítulo se presentan los resultados obtenidos tras la implementación de la arquitectura híbrida propuesta. El análisis se estructura de acuerdo con las distintas fases del sistema —aprendizaje no supervisado, modelado secuencial con Transformer y mecanismos de explicabilidad— evaluando su desempeño en función de las diferentes ventanas temporales definidas. Estos resultados permiten valorar la viabilidad técnica del enfoque adoptado y establecer una base sólida para la optimización y consolidación final del modelo en la siguiente fase del proyecto.

6.1. Resultados de Autoencoder.

El entrenamiento del autoencoder se realizó bajo un esquema no supervisado, cuyo objetivo principal consiste en aprender una representación latente comprimida de los datos que preserve la mayor cantidad de información estructural posible, para poder validar su funcionalidad se analiza la fidelidad de reconstrucción junto con visualizaciones de las representaciones vectoriales.

6.1.1. Fidelidad de reconstrucción.

Para evaluar la capacidad de reconstrucción del modelo se utilizó la función de pérdida Huber Loss que mide la discrepancia entre el vector original de entrada x y su reconstrucción x' generada por el autoencoder. A mayor diferencia entre ambos, mayor será el valor de la pérdida, indicando una menor fidelidad en la reconstrucción.

La siguiente imagen muestra la evolución de la pérdida de reconstrucción tanto en el conjunto de entrenamiento como en el conjunto de validación a lo largo de las épocas.

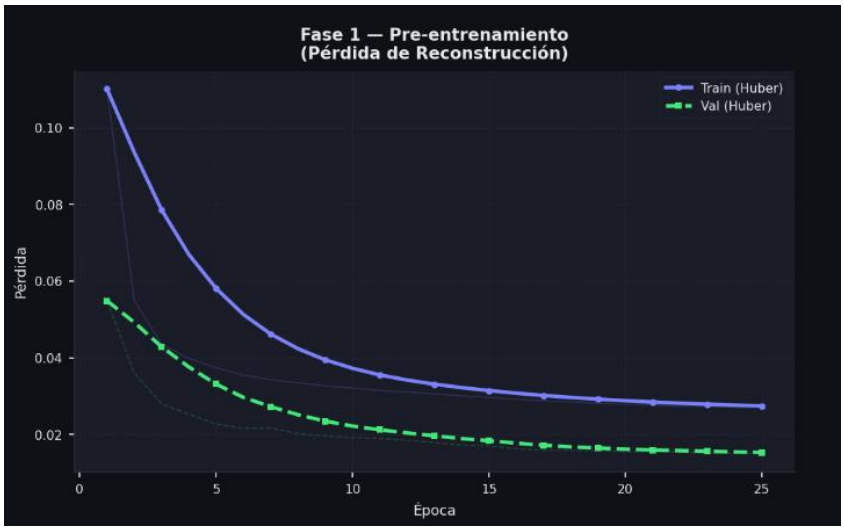

Ilustración 13 Evolución de huber loss durante entrenamiento de autoencoder

El rápido descenso inicial de ambas curvas y su progresiva estabilización indican que el autoencoder aprende eficazmente una representación latente y converge sin signos de sobreajuste, al no presentar divergencia entre ellas. Además, que la pérdida de validación se mantenga ligeramente por debajo de la de entrenamiento es un comportamiento esperado debido a mecanismos de regularización como Dropout o Batch Normalization. Mientras que en la fase de entrenamiento estas técnicas introducen ruido y penalizan la red, durante la validación se desactivan, lo que permite al modelo generar reconstrucciones más consistentes y reportar un error menor.

Una vez completada la fase de preentrenamiento se inicia la fase de entrenamiento conjunto, en la que se incorpora el término adicional correspondiente al objetivo de clustering (divergencia KL): por un lado, busca la preservación de la información original y, por otro, la reorganización estructural del espacio latente en torno a centroides probabilísticos. La dinámica de este proceso puede observarse en la siguiente figura, donde se muestra la evolución

de la pérdida asociada al objetivo de clustering durante el entrenamiento conjunto.

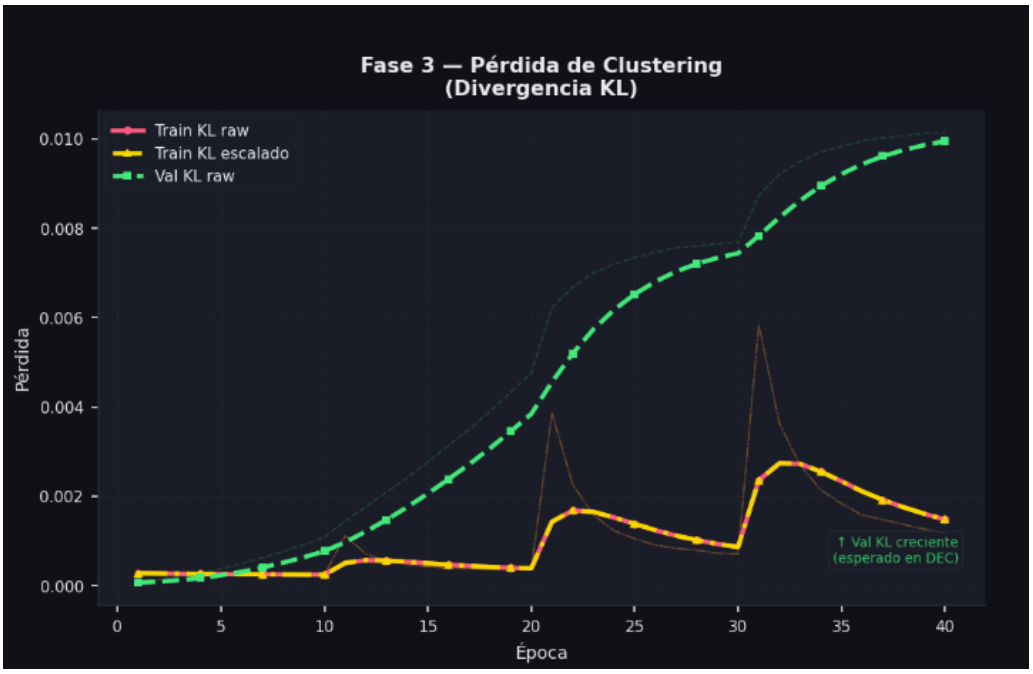

Ilustración 14 Evolución de perdidas en la fase de entrenamiento conjunto

Durante la Fase 3, se observa que la métrica de objetivo (o divergencia KL) aumenta. Como se espera en la arquitectura DEC, esto es un artefacto matemático del endurecimiento progresivo de las probabilidades de asignación a los clústeres en la distribución objetivo (P). Para demostrar que este comportamiento interno se traduce en una mejora real del modelo, evaluamos la topología del espacio latente utilizando métricas independientes de la pérdida: el coeficiente de Silhouette y el índice Davies-Bouldin. Como se observa en la siguiente figura, ambas métricas mejoran significativamente durante esta misma fase, confirmando el éxito del agrupamiento.

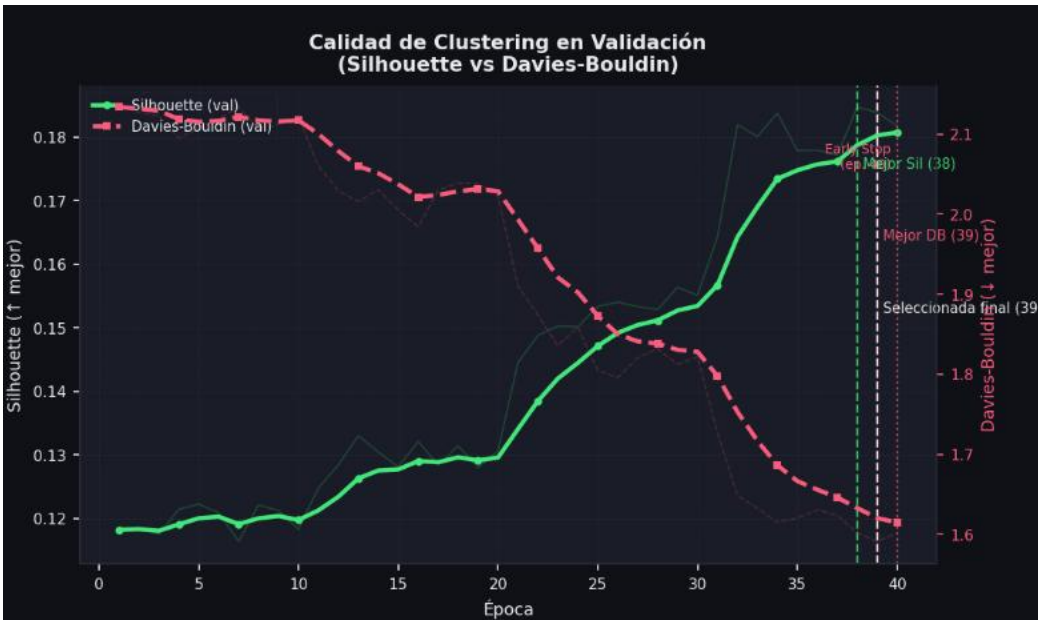

Ilustración 15 Evolución de Sillouette y Davies-Bounldir en la fase 3

El análisis de las métricas de validación confirma una mejora continua y significativa en la calidad del agrupamiento a lo largo de esta fase. Específicamente, a partir de la época 20 se observa una fuerte evolución conjunta: el coeficiente de Silhouette experimenta un incremento constante, lo que indica una mayor cohesión y densidad dentro de los clústeres, mientras que el índice Davies-Bouldin desciende de forma pronunciada, reflejando una mejor separación y menor superposición entre ellos. La convergencia de ambos indicadores hacia sus valores óptimos en las etapas finales del entrenamiento valida la eficacia del modelo para estructurar el espacio latente, justificando la selección de la época 39 como el punto de equilibrio ideal.

A continuación, se procede a analizar de forma conjunta la evolución de la pérdida de reconstrucción y del objetivo monitorizado en validación durante la fase de entrenamiento conjunto. Esta comparación permite evaluar el equilibrio entre la fidelidad reconstructiva y el término de clustering a lo largo de las épocas.

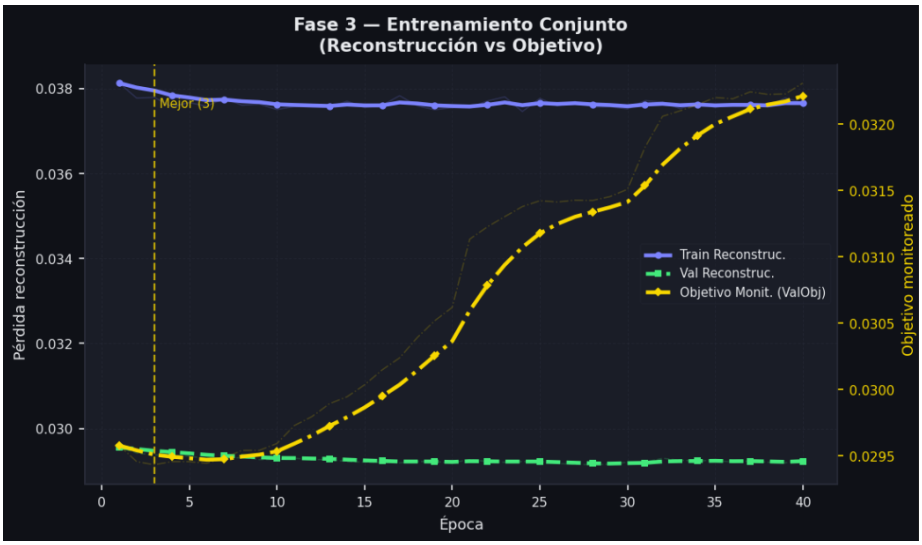

Ilustración 16 Balance entre reconstrucción y objetivo de clustering en fase conjunta

Durante el entrenamiento conjunto se observa que la pérdida de reconstrucción, tanto en entrenamiento como en validación, se mantiene prácticamente estable tras la fase de preentrenamiento, lo que indica que la incorporación del término de clustering no degrada la capacidad reconstructiva del autoencoder. Paralelamente, el objetivo monitorizado (ValObj) muestra una tendencia creciente tras las primeras épocas, reflejando el reajuste dinámico introducido por la optimización conjunta. La mejor configuración se alcanza tempranamente (época 3), a partir de la cual el incremento progresivo del objetivo sugiere que el modelo comienza a priorizar la estructuración latente sin aportar mejoras adicionales en validación. Este comportamiento respalda la aplicación de early stopping y confirma la existencia de un punto de equilibrio óptimo entre fidelidad reconstructiva y organización del espacio latente.

6.1.2. Representaciones vectoriales (Embeddings).

En la siguiente ilustración se muestra una representación comparativa del funcionamiento del autoencoder en tres niveles: (1) el espacio original proyectado mediante PCA, (2) el espacio latente aprendido por el modelo y (3) la reconstrucción nuevamente proyectada en el espacio PCA. Las filas corresponden a distintas ventanas temporales (W10 y W15), lo que permite analizar la estabilidad estructural del mapeo aprendido conforme avanza el curso.

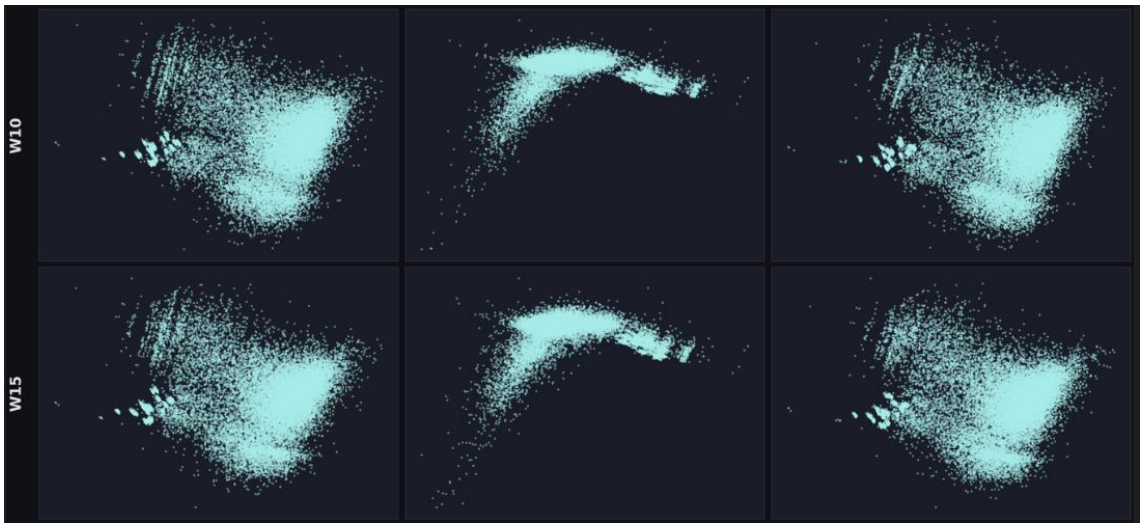

Ilustración 17 Representación de datos de estudiantes en el espacio original, latente y reconstrucción - Autoría Propia - Apéndice I - Reportes -AE

Puede observarse cómo, en el espacio latente, las representaciones se compactan y reorganizan en estructuras más densas y diferenciadas, reflejando el efecto combinado de la pérdida de reconstrucción y del término de clustering. A su vez, la proyección de la reconstrucción conserva la forma global del espacio original, evidenciando que la compresión no implica una pérdida significativa de estructura. Este comportamiento confirma que el autoencoder no opera únicamente como un mecanismo de reducción dimensional, sino como un generador de embeddings estructuralmente coherentes y optimizados para la segmentación, favoreciendo la emergencia de perfiles de aprendizaje más definidos y matemáticamente separables.

6.2. Resultados de clustering.

Tal y como se justificó en el apartado 4.2.2, la arquitectura seleccionada para la fase de agrupamiento combina técnicas de reducción de dimensionalidad con el Modelo de Mixtura Gaussiana (GMM), priorizando aquellas configuraciones más coherentes con el enfoque probabilístico e híbrido del sistema propuesto.

Con el objetivo de mantener la memoria concisa y centrada en las configuraciones de mayor interés experimental, se omiten los resultados correspondientes a combinaciones alternativas como PCA + K-Means o Autoencoder + K-Means. En esta sección se analizan exclusivamente los resultados obtenidos mediante Autoencoder + GMM, al ser las configuraciones alineadas con la arquitectura final integrada en el pipeline predictivo.

Los resultados que se analizarán a continuación se encuentran en la carpeta de reports/clustering (Véase Apéndice I). Se procede a analizar la distribución de resultados académicos dentro de cada grupo identificado. Este análisis permite evaluar si los grupos generados presentan coherencia semántica y diferenciación real en términos de rendimiento y abandono, más allá de las métricas internas de calidad del clustering.

La siguiente figura muestra la distribución porcentual de los resultados académicos dentro de cada uno de los perfiles identificados por el modelo de clustering. Para cada grupo se representa la proporción de estudiantes que obtienen aprobación, suspenso o abandono, diferenciando entre el conjunto de validación y el conjunto de prueba. Esta visualización permite analizar no solo la coherencia interna de los perfiles descubiertos, sino también su estabilidad fuera de la muestra de entrenamiento, evaluando si la segmentación captura patrones estructurales consistentes en el comportamiento académico.

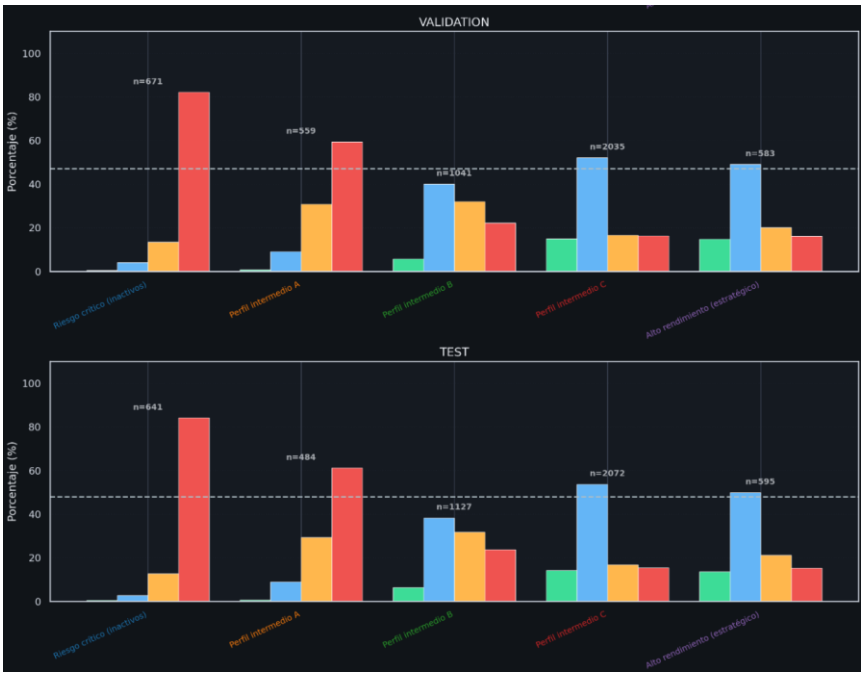

Ilustración 18 Agrupaciones de estudiantes por grupos

Utilizando únicamente la información de las primeras cinco semanas, el modelo ya logra segmentar eficazmente los perfiles según su nivel de riesgo. Destaca especialmente el grupo de "Riesgo crítico (inactivos)" (clúster 0), que concentra más del 80% de los abandonos tanto en validación como en test, lo que demuestra una notable capacidad predictiva como sistema de alerta temprana. Además, los perfiles intermedios exhiben un gradiente coherente y progresivo desde las tasas de abandono hacia las de aprobación, confirmando la solidez del ordenamiento latente aprendido por el algoritmo.

Sin embargo, en esta fase inicial el perfil de "Alto rendimiento (estratégico)" aún no presenta una separación concluyente, indicando que, con datos limitados, el modelo es más preciso detectando el riesgo de abandono que confirmando el éxito académico definitivo. Para evaluar la consolidación de estos grupos, a continuación, se analizan los resultados de la semana 20 (W20); en esta etapa avanzada, la mayor acumulación de información conductual y evaluativa debería reflejar una diferenciación mucho más nítida y definitiva entre todos los clústeres.

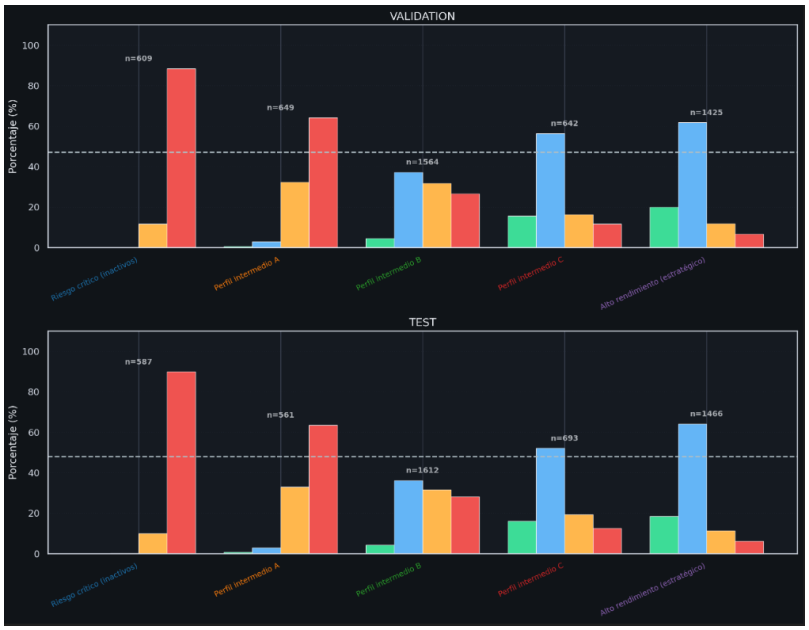

Ilustración 19 Distribución de resultados académicos por perfil en la semana 20 (Validación y Test)

En la semana 20 (W20), la mayor acumulación de datos permite una consolidación mucho más nítida de los perfiles. Por un lado, el grupo de "Riesgo crítico" (clúster 0) reafirma su estabilidad manteniendo una tasa de abandono cercana al 90%, lo que confirma la fiabilidad de la alerta temprana. Por otro lado, a diferencia de lo ocurrido en la semana 5, el perfil de "Alto rendimiento" logra ahora una diferenciación clara, concentrando más del 60% de aprobados con un abandono mínimo. Esta evolución demuestra que, al incorporar más evidencias a lo largo del curso, el modelo aumenta significativamente su poder discriminativo.

Para dotar de interpretabilidad a estos resultados, la sección de Inteligencia Artificial Explicable (XAI) enfocada en clustering empleará una matriz Z-Effect. Esta herramienta analiza la desviación de cada variable respecto a la media global, permitiendo identificar los patrones conductuales que definen a los perfiles extremos y revelando los matices específicos que justifican las agrupaciones intermedias.

6.3. Resultados en Transformer.

Tal y como se ha expuesto anteriormente, la arquitectura Transformer permite adaptar de forma flexible la definición del problema de clasificación, pudiendo abordarse como tarea binaria, trinaría o cuaternaria en función de la configuración de las etiquetas. En los siguientes epígrafes se analizará cada una de estas formulaciones de manera independiente.

6.3.1. Clasificación binaria: Aprobar vs Abandonar.

En esta configuración se consideran únicamente dos clases diferenciadas en el entrenamiento: por un lado, los estudiantes que aprueban o alcanzan distinción, y por otro, aquellos que abandonan el curso (withdrawn). En consecuencia, los estudiantes con resultado de suspenso quedan excluidos de esta fase de modelado, con el objetivo de simplificar la frontera de decisión y centrar el análisis en la discriminación temprana entre éxito académico y abandono.

Métrica	W=5	W=10	W=15	W=20	W=25
Accuracy	0.792	0.847	0.898	0.942	0.955
ROC (One-vs-Rest)	0.828	0.899	0.936	0.965	0.986
Balanced Accuracy	0.756	0.838	0.874	0.922	0.946
F1-Score.	0.659	0.764	0.829	0.901	0.926
Loss.	0.038	0.029	0.023	0.016	0.014
Precision	0.656	0.719	0.845	0.932	0.930
Recall	0.6633	0.8156	0.8128	0.8720	0.9230

Tabla 1 Métricas obtenidas en clasificación binaria (Éxito vs Abandono)

El análisis comparativo de las distintas ventanas temporales evidencia una mejora progresiva y sostenida del rendimiento predictivo conforme avanza el curso. Desde la semana 5 hasta la semana 25 se observa un incremento consistente en todas las métricas clave, especialmente en AUC, F1-Score y

balanced accuracy, junto con una reducción estable de la pérdida. Este comportamiento confirma que el modelo es capaz de aprovechar la acumulación de información conductual y evaluativa para refinar la discriminación entre perfiles, mejorando tanto la capacidad de clasificación global como el equilibrio entre clases.

No obstante, resulta especialmente relevante que, incluso en la semana 5, el modelo ya alcanza un nivel de desempeño razonable (AUC > 0.82), lo que sugiere capacidad de detección temprana de patrones de riesgo. A medida que se incorporan más semanas de actividad, la separación entre perfiles se consolida hasta alcanzar valores muy elevados en la semana 25 (AUC ≈ 0.99), reflejando una estructura latente altamente informativa. En conjunto, estos resultados indican que el sistema no solo mejora con mayor evidencia temporal, sino que mantiene utilidad práctica desde fases iniciales del curso, aspecto clave para aplicaciones de alerta temprana en contextos educativos.

6.3.2. Otros tipos de clasificación.

De forma complementaria a la formulación binaria principal (Aprobados vs. Abandonos), se evaluaron configuraciones de clasificación trinaría y cuaternaria con el objetivo de analizar el comportamiento del modelo bajo escenarios de mayor granularidad. Los resultados completos pueden consultarse en el Apéndice III, donde se detallan todas las métricas por ventana temporal. De manera general, se observa una evolución coherente y progresiva del rendimiento en todos los casos: las métricas mejoran sistemáticamente conforme aumenta la información disponible (W=5 $\rightarrow$ W=25) y la función de pérdida desciende de forma estable, lo que confirma la consistencia del entrenamiento.

No obstante, el incremento en el número de clases introduce una mayor complejidad en la frontera de decisión, reduciendo el rendimiento global respecto

a la formulación binaria. En el escenario trinario se aprecia una caída moderada en accuracy y F1-macro, mientras que en la configuración cuaternaria la degradación es más acusada, especialmente en balanced accuracy y métricas macro, reflejando un mayor solapamiento entre perfiles académicos intermedios.

En conjunto, estos resultados evidencian que, aunque el modelo mantiene estabilidad estructural, su desempeño óptimo se alcanza en la formulación binaria (Éxito vs Abandono), siendo esta la más adecuada para el objetivo predictivo planteado en el presente trabajo.

6.4. IA Explicable (XAI).

6.4.1. IA Explicativa en el clustering.

Aunque el modelo de clustering segmenta perfiles con coherencia estadística, es necesario interpretar qué variables explican su diferenciación. Para ello, se incorpora un análisis de IA Explicable (XAI) que permite caracterizar cada clúster en términos conductuales y dotarlo de significado aplicado.

Con este fin, se utiliza la matriz Z-Effect, que recoge las 30 variables más discriminativas por perfil. Los valores representan efectos estandarizados respecto a la media global, permitiendo identificar qué dimensiones están sobrerrepresentadas o infrarepresentadas en cada grupo y facilitando la comparación estructural entre perfiles.conductuales explican la pertenencia a cada clúster.

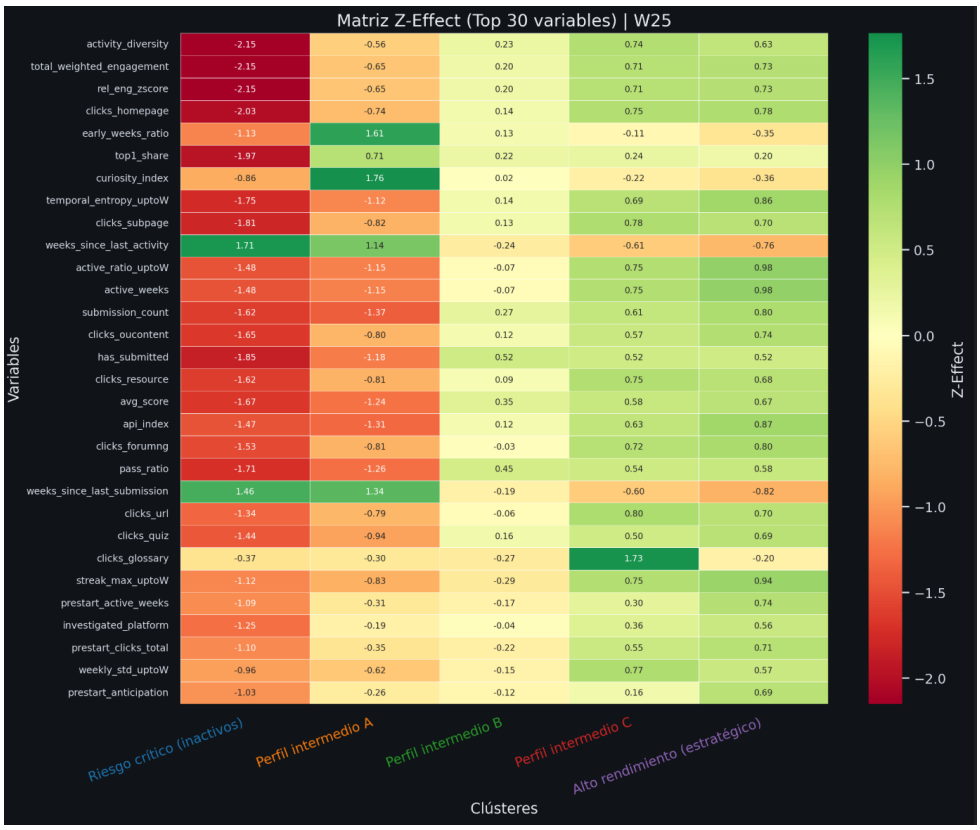

Ilustración 20 Matriz Z-Effect de las 30 variables más discriminativas por clúster (W25)

El análisis de la matriz Z-Effect revela contrastes estructurales muy definidos y confirma que los clústeres no son categorías aisladas, sino posiciones dentro de un continuo conductual estructurado principalmente por la regularidad temporal, la recencia de la actividad y el rendimiento evaluativo (api_index, pass_ratio). Esta diferenciación permite caracterizar los siguientes perfiles y definir intervenciones adaptativas:

- Riesgo crítico (inactivos): Evidencia una fuerte desvinculación, con valores mínimos en interacción y regularidad (active_weeks, activity_diversity) y máximos en inactividad prolongada (weeks_since_last_activity). Requiere intervenciones tempranas centradas en la reactivación y el seguimiento continuo.
- Alto rendimiento (estratégico): Destaca por su participación sostenida, intensidad de interacción y éxito académico. Su rasgo más distintivo es el comportamiento proactivo previo al inicio del curso (prestart_active_weeks, prestart_anticipation). Son candidatos ideales para estrategias de enriquecimiento o mentoría.
- Perfiles intermedios (A, B y C): Presentan configuraciones híbridas que se benefician de estrategias de acompañamiento para consolidar la regularidad y evitar el abandono. El Perfil A concentra su actividad al inicio (early_weeks_ratio), el Perfil B mantiene una interacción moderada, y el Perfil C se aproxima al alto rendimiento, aunque sin consolidar su nivel de anticipación.

7. Conclusiones.

Desde una perspectiva aplicada, estos resultados sugieren líneas de intervención diferenciadas. El perfil de Riesgo crítico requiere acciones tempranas centradas en reactivación y seguimiento de continuidad, priorizando la reducción de periodos de inactividad prolongada. Los perfiles intermedios podrían beneficiarse de estrategias de acompañamiento que fomenten regularidad y consolidación evaluativa, evitando la transición hacia abandono. En cambio, el perfil de Alto rendimiento puede ser objeto de estrategias de enriquecimiento o mentoría, aprovechando su comportamiento estratégico y proactivo. De este modo, la interpretación explicativa del clustering no solo valida el modelo estadísticamente, sino que proporciona una base operativa para intervenciones educativas adaptativas.

8. Referencias

Al-Tameemi, G., Xue, J., Ali, I. H., & Ajit, S. (2024). A Hybrid Machine Learning Approach for Predicting Student Performance Using Multi-class Educational Datasets. *Procedia Computer Science, 238*, 888-895. doi:10.1016/j.procs.2024.06.108

El Ghali, M., Atouf, I., El Guemmat, K., Broumi, S., & Talea, M. (2025). ENHANCING E-LEARNING THROUGH STRATEGIC STUDENT SEGMENTATION: INSIGHTS FROM THE OULAD DATABASE . *Theoretical and Applied Information Technology, 103*(4), 1290-1299.

Firat, M. (2025). Comparative Analysis of Random Forest vs XGBoost Machine Learning Algorithms for Predicting ODL Student Success.

Gamage, S. H., Ayres, J. R., & Behrend, M. B. (Enero de 2022). A systematic review on trends in using Moodle for teaching and learning. *International Journal of STEM Education, 9*(9), 1-24. doi:https://doi.org/10.1186/s40594-021-00323-x

Gašević, D., Dawson, S., Rogers, T., & Gasevic, D. (2016). Learning analytics should not ignore instructional conditions. *Computers & Education*, 53-66.

Hasan, R., Palaniappan, S., Mahmood, S., Abbas, A., Sarker, K. U., & Sattar, M. U. (2020). Predicting Student Performance in Higher Educational Institutions Using Video Learning Analytics and Data Mining Techniques. *Applied Sciences, 10*(11), 3894. doi:10.3390/app10113894

Instituto Andaluz Interuniversitario en Ciencia de Datos e Inteligencia Computacional (DASCI). (s.f.). *DASCI.* Recuperado el 2026, de Minería

de datos educativos y análisis del aprendizaje: https://dasci.es/linea-investigacion/mineria-de-datos-educativos/

Junejo, N. U., Nawaz, M. W., Huang, Q., Dong, X., Wang, C., & Zheng, G. (2024). Accurate Multi-Category Student Performance Forecasting at Early Stages of Online Education Using Neural Networks. *arXiv preprint arXiv:2412.05938*, 1-27.

Khosravi, H., Buckingham Shum, S., Chen, G., Conati, C., Tsai, Y.-S., Kay, J., . . . Gašević, D. (2022). Computers and Education: Artificial Intelligence. *Computers and Education: Artificial Intelligence, 3*. Obtenido de https://doi.org/10.1016/j.caeai.2022.100074

Kusumawardani, S. S., & Alfarozi, S. A. (2023). Transformer Encoder Model for Sequential Prediction of Student Performance Based on Their Log Activities. *11*, págs. 18960-18971. IEEE. doi:10.1109/ACCESS.2023.3246122

Kuzilek, J., Hlosta, M., & Zdrahal, Z. (2017). Open University Learning Analytics dataset. *Scientific Data, 4*(170171).

Luo, H., Yu, Q., Xie, S., Wu, H., & Xu, K. (2024). GLU-Transformer for Predicting Withdrawal in VLE. *ICBAR '24: Proceedings of the 2024 4th International Conference on Big Data, Artificial Intelligence and Risk Management*, 835-840.

Papadogiannis, I., Wallace, M., & Karountzou, G. (2024). Educational Data Mining: A Foundational Overview. *Encyclopedia, 4*(4), 1644-1664.

Parlamento Europeo y Consejo de la Unión Europea. (2024). *Reglamento (UE) 2024/1689 por el que se establecen normas armonizadas sobre*

inteligencia artificial (Ley de Inteligencia Artificial). Bruselas. Obtenido de http://data.europa.eu/eli/reg/2024/1689/o

Sejnowski, T. J. (2020). The unreasonable effectiveness of deep learning in artificial intelligence. *Proc Natl Acad Sci U S A.* doi:10.1073/pnas.1907373117

Soepriyanto, Y., Nugroho, R. P., Nahri, M. H., Kesuma, D. W., & Setiasih, M. (2025). From logs to insights: A comprehensive framework for data-driven learning insights. *Jurnal Inovasi Teknologi Pendidikan, 12*(1), 40-49. doi:https://doi.org/10.21831/jitp.v12i1.77432

Torkhani, W., & Rezgui, K. (2025). OULAD MOOC Student Performance. (págs. 228-241). Atlantis Press. doi:10.2991/978-94-6463-654-3_18

Apéndice I: Repositorio de Código y Reproducibilidad

Acceso al Repositorio Todo el código fuente, los notebooks de exploración (EDA) y los scripts de entrenamiento desarrollados para este Trabajo de Fin de Máster se encuentran alojados en un repositorio público de GitHub para garantizar la transparencia y reproducibilidad de los resultados.

Atributo	Detalle
URL del Repositorio	https://github.com/MichaeLaudrup/Artificial_Intelligence_Msc/tree/main/TFM_education_ai_analytics`
Plataforma	GitHub
Lenguaje Principal	Python 3.12+
Librerías Clave	Pandas, TensorFlow/Keras, Scikit-learn, SHAP
Licencia	MIT License (Código abierto)
Código QR para acceso directo	

Tabla 2 Ficha técnica de repositorio código fuente

Estructura del Repositorio: El proyecto sigue una arquitectura modular inspirada en los estándares de la industria para ciencia de datos, lo que garantiza la separación de preocupaciones y facilita la reproducibilidad del experimento. A continuación, se detalla la jerarquía de directorios y la función de sus componentes principales:

root/: Directorio principal del proyecto que contiene los archivos de configuración y automatización.

- data/: Almacén central de datos, organizado por etapas de madurez:
    - raw/: Copias inalteradas de los datos originales (OULAD).
    - interim/: Datos transformados durante los procesos de limpieza intermedios.
    - processed/: Conjuntos de datos finales, anonimizados y listos para alimentar los modelos de IA.
    - external/: Datos complementarios de terceras fuentes utilizados en el análisis.
- notebooks/: Cuadernos interactivos (.ipynb) utilizados para la experimentación inicial.
    - data_exploration.ipynb: Documenta el análisis exploratorio (EDA), visualizaciones estadísticas y la validación de hipótesis iniciales.
- educational_ai_analytics/: Núcleo del código fuente desarrollado como un paquete modular de Python:
    - dataset.py: Funciones para la extracción y carga eficiente de los datos.
    - features.py: Implementación de la ingeniería de variables y transformadores.
    - modeling/: Scripts dedicados al entrenamiento, validación cruzada y serialización de los modelos.
    - plots.py: Motor de visualización personalizado para la generación de gráficas presentadas en este trabajo.
    - config.py: Definición de hiperparámetros, rutas del sistema y constantes globales.
- models/: Directorio destinado a almacenar los pesos de los modelos entrenados y los archivos de metadatos asociados.
- reports/: Resultados del análisis.
    - figures/: Gráficas y representaciones visuales exportadas en alta resolución para la memoria del TFM.

- references/: Documentación técnica, diccionarios de datos y manuales de las fuentes de información utilizadas.
- Makefile: Archivo de automatización que permite replicar todo el flujo de trabajo (limpieza, procesado y entrenamiento) mediante comandos simples (ej. make data, make models).
- pyproject.toml: Especificación de la versión de Python (3.12+) y lista exhaustiva de dependencias necesarias para recrear el entorno virtual.

Apendice II: Tablas de características

Características del autoencoder.

Característica	Descripción
imd_band	Índice de Privación Múltiple (proxy de nivel socioeconómico), mapeado numéricamente.
age_band	Rango de edad del estudiante categorizado numéricamente.
highest_education	Nivel educativo previo del estudiante (escala ordinal).
num_of_prev_attempts	Número de veces que el alumno ha intentado superar el módulo previamente.
studied_credits	Carga académica total (créditos) matriculada simultáneamente.
region_encoded	Región geográfica de residencia del estudiante, codificada numéricamente.
investigated_platform	Variable binaria (0/1) que indica si el alumno accedió a la plataforma antes de iniciar el curso.
prestart_clicks_total	Volumen total de clics registrados antes del inicio oficial.
prestart_active_days	Número total de días distintos en los que hubo actividad previa al inicio.
prestart_active_weeks	Número total de semanas distintas con actividad antes de arrancar.
prestart_earliest_day	Día exacto (en negativo) en el que el alumno realizó su primera conexión histórica.
prestart_anticipation	Valor absoluto del primer día de conexión (días de antelación al inicio).
prestart_intensity	Ratio de intensidad pre-curso (clics totales divididos entre días activos).
clicks_[tipo_actividad]	18 variables independientes que acumulan los clics normalizados por cada tipo de recurso (ej. clicks_forumng, clicks_homepage, clicks_quiz, clicks_oucontent, etc.).
total_weighted_engag	Compromiso acumulado aplicando ponderaciones

ement	(ej. +peso a cuestionarios, -peso a lecturas).
rel_eng_zscore	Proxy del compromiso (total_weighted_engagement), normalizado (Z-score) por curso.
last_week_clicks_weighted	Volumen de compromiso ponderado generado específicamente en la última semana evaluada.
momentum_2w	Aceleración del esfuerzo: compara la actividad de las últimas 2 semanas frente a las 2 previas.
effort_slope	Pendiente de regresión lineal sobre los clics semanales (tendencia general de esfuerzo).
temporal_entropy_uptoW	Entropía de la distribución temporal (valores altos = constancia; bajos = picos esporádicos).
weekly_std_uptoW	Desviación estándar de los clics semanales (mide la volatilidad en el ritmo de estudio).
activity_diversity	Cantidad de tipos de recursos diferentes (foros, wikis, pdfs) que el estudiante ha utilizado.
curiosity_index	Ratio entre la diversidad de recursos utilizados y el número de semanas activas.
top1_share	Proporción de la actividad total concentrada exclusivamente en el recurso más utilizado.
prestart_ratio	Proporción de los clics totales del curso que ocurrieron antes del Día 0.
early_weeks_ratio	Proporción del esfuerzo total concentrado en las primeras 4 semanas del curso.
first_active_week	Semana exacta en la que el estudiante registró su primera interacción post-inicio.
active_weeks	Número total de semanas en las que el alumno interactuó con el VLE.
active_ratio_uptoW	Porcentaje de semanas con actividad respecto al total de semanas transcurridas.
streak_max_uptoW	Racha máxima histórica de semanas consecutivas con actividad.
streak_final	Racha de semanas consecutivas de actividad que llega viva hasta la semana actual (W-1).
weeks_since_last_activity	Semanas de inactividad transcurridas desde el último clic (indicador clave de riesgo).
has_submitted	Variable binaria (0/1) que indica si el alumno ha

	entregado al menos una actividad.
submission_count	Número total de evaluaciones entregadas hasta la fecha de corte.
avg_score	Calificación media obtenida en las tareas entregadas (0-100).
score_std	Desviación estándar de las calificaciones (estabilidad del rendimiento del alumno).
pass_ratio	Porcentaje de entregas que superan la nota mínima de aprobación (40%).
late_ratio	Porcentaje de entregas realizadas fuera de la fecha límite oficial.
score_slope	Tendencia de las calificaciones (pendiente ascendente o descendente en el rendimiento).
weeks_since_last_submission	Semanas transcurridas desde que el alumno realizó su última entrega evaluable.
api_index	Índice compuesto que premia conjuntamente buenas notas y volumen de entregas.

Tabla 3 Características de entrada al autoencoder

Apéndice III: Métricas de entrenamiento en Transformers.

- Resultados en clasificación binaria (Existo vs Fracaso)

Métrica	W=5	W=10	W=15	W=20	W=25
Accuracy	0.7403	0.8000	0.8334	0.8750	0.8946
ROC (One-vs-Rest)	0.8083	0.8721	0.9085	0.9396	0.9578
Balanced Accuracy	0.7345	0.7938	0.8294	0.8704	0.8915
F1-Score.	0.7008	0.7669	0.8116	0.8568	0.8818
Loss.	0.0481	0.0409	0.0348	0.0289	0.0240
Precision.	0.7481	0.8321	0.8508	0.9112	0.9171
Recall.	0.6591	0.7111	0.7759	0.8086	0.8491

Tabla 4 Métricas de entrenamiento del transformer clase binaria (Exito vs Fracaso)

- Resultados clasificación trinaria (éxito, suspenso, abandono)

Métrica	5	10	15	20	25
Accuracy	0.6381	0.6853	0.7098	0.7317	0.7631
AUC (OvR)	0.7446	0.8045	0.8375	0.8698	0.8924
Balanced Accuracy	0.5330	0.6088	0.6416	0.6778	0.6978
F1 Macro	0.5379	0.6156	0.6450	0.6743	0.6930
F1 Weighted	0.6133	0.6794	0.7065	0.7338	0.7562
Precision Macro	0.5629	0.6260	0.6493	0.6715	0.6932
Recall Macro	0.5330	0.6088	0.6416	0.6778	0.6978
Precision Weighted	0.6077	0.6768	0.7040	0.7364	0.7529
Recall Weighted	0.6381	0.6853	0.7098	0.7317	0.7631
Loss	0.0946	0.0795	0.0708	0.0603	0.0506

Tabla 5 Resultados del transformer en clasificación binaria.

- Clasificación cuaternaria (Éxito, abandono, suspenso, sobresaliente).

Métrica	5	15	20	25
Accuracy	0.4674	0.5666	0.6212	0.6444
Balanced Accuracy	0.4509	0.5762	0.6116	0.6403
AUC (OvR)	0.7318	0.8191	0.8506	0.8707
F1 Macro	0.4464	0.5510	0.5913	0.6217
F1 Weighted	0.4750	0.5736	0.6215	0.6508
Precision Macro	0.4683	0.5452	0.5810	0.6140
Recall Macro	0.4509	0.5762	0.6116	0.6403
Precision Weighted	0.5075	0.5929	0.6275	0.6649
Recall Weighted	0.4674	0.5666	0.6212	0.6444
Top-2 Accuracy	0.7773	0.8916	0.9318	0.9509
Loss	0.1113	0.0833	0.0664	0.0561

Anexos I: Estructura y diccionario de datos OULAD.

- Tabla I: Dimensión del estudiante (students_processed.csv)

Esta tabla maestra consolida la información estática y demográfica, actuando como el perfil base de cada alumno matriculado.

Campo	Descripción
code_module	Identificador del curso (ej. AAA, BBB).
code_presentation	Semestre en el que se imparte (ej. 2013J, 2014B).
id_student	ID único anonimizado del estudiante.
gender	Género del estudiante (M/F).
region	Región geográfica de residencia en UK.
highest_education	Nivel educativo previo (ej. A Level, HE Qualification).
imd_band	Índice de Privación Múltiple (nivel socioeconómico; % más bajo indica mayor pobreza).
age_band	Rango de edad (0-35, 35-55, 55<=).
num_of_prev_attempts	Número de veces que el estudiante ha intentado este módulo previamente.
studied_credits	Carga total de créditos que el estudiante cursa actualmente.
disability	Indicador de discapacidad declarada (Y/N).
final_result	Variable Objetivo: Pass, Distinction, Fail, Withdrawn.
date_registration	Día relativo al inicio del curso en que se formalizó la matrícula.
date_unregistration	Día relativo en que se produjo la baja (si aplica).

Tabla 6 Descripción de datos de estudiantes

- Tabla II: Dimensión de Rendimiento (assessments_processed.csv)

Contiene los resultados cuantitativos de los hitos evaluativos, permitiendo trazar la evolución académica del alumno.

Campo	Descripción
id_assessment	ID único de la prueba específica.
id_student	ID del estudiante.
date_submitted	Día relativo al inicio del curso en que se realizó la entrega.
is_banked	Indicador de si la nota ha sido convalidada de una presentación anterior.
score	Calificación obtenida (escala 0-100).
assessment_type	Tipología: TMA (Corrección por tutor), CMA (Corrección automática), Exam.
date	Fecha límite oficial de entrega (*Deadline*).
weight	Ponderación de la evaluación en la nota final del curso.

Tabla 7 Descripción de entrega de actividades

Tabla III: Dimensión de Comportamiento (interactions_processed.csv)

Representa el log de actividad diario en el Entorno Virtual de Aprendizaje (VLE). Dada su granularidad, constituye la fuente principal para el modelado secuencial.

Campo	Descripción
code_module	Identificador del curso (ej. AAA, BBB).
code_presentation	Semestre en el que se imparte (ej. 2013J, 2014B).
id_student	ID único anonimizado del estudiante.
gender	Género del estudiante (M/F).
region	Región geográfica de residencia en UK.
highest_education	Nivel educativo previo (ej. A Level, HE Qualification).
imd_band	Índice de Privación Múltiple (nivel socioeconómico; % más bajo indica mayor pobreza).
age_band	Rango de edad (0-35, 35-55, 55<=).

num_of_prev_attempts	Número de veces que el estudiante ha intentado este módulo previamente.
studied_credits	Carga total de créditos que el estudiante cursa actualmente.
disability	Indicador de discapacidad declarada (Y/N).
final_result	Variable Objetivo: Pass, Distinction, Fail, Withdrawn.
date_registration	Día relativo al inicio del curso en que se formalizó la matrícula.
date_unregistration	Día relativo en que se produjo la baja (si aplica).

Tabla 8 Descripción datos de interacción de estudiantes